

Aprendizaje y Clasificación Automática con R

Un enfoque práctico con datos abiertos reales

MI Jesús Gilberto Rodríguez Escobedo

2026-07-14

Índice general

Presentación	1
Aprendizaje y Clasificación Automática con R	1
Un enfoque práctico con datos abiertos reales	1
Objetivo general	1
Público al que va dirigido	1
Software utilizado	1
Cuaderno completo de Google Colab	2
Estructura actual del libro	2
Nota para el lector	2
Fuente principal de los datos	2
Licencia, autoría y forma de citar	3
Autor	3
Licencia	3
Cita sugerida	3
Uso educativo	3
Cuaderno completo para Google Colab	4
¿Qué contiene el cuaderno?	4
Cómo utilizarlo	4
Diferencias entre el libro web y el cuaderno	5
1 ¿Qué es el aprendizaje automático?	6
1.1 Objetivos del capítulo	6
1.2 Introducción	6
1.3 Programación tradicional contra aprendizaje automático	6
1.4 Variables predictoras y variable respuesta	7
1.5 Tipos principales de aprendizaje automático	7
1.6 Notación básica	8
1.7 Primer ejemplo visual en R	8
1.8 División entre entrenamiento y prueba	9
1.9 Primer clasificador basado en una regla	10
1.10 Conclusión	10
2 Preparación de datos reales	11
2.1 Objetivos del capítulo	11
2.2 Introducción	11
2.3 Descargar la base ATUS desde el INEGI	11
2.4 Cómo citar los datos	12
2.5 Cargar paquetes	13
2.6 Localizar el archivo	13
2.7 Leer el archivo CSV	13
2.8 Primer vistazo a los datos	14
2.9 Normalizar los nombres de variables	15

2.10	Seleccionar variables de interés	15
2.11	Crear la variable respuesta	16
2.12	Preparar las variables finales	17
2.13	Guardar la base preparada	17
2.14	Lista de comprobación	18
2.15	Resumen del capítulo	18
3	Análisis exploratorio de datos	19
3.1	Objetivos del capítulo	19
3.2	Cargar paquetes y base	19
3.3	Preparar variables	19
3.4	Distribución de la variable respuesta	20
3.5	Accidentes por mes	21
3.6	Accidentes por hora del día	22
3.7	Tipos de accidente más frecuentes	23
3.8	Proporción por tipo de accidente	24
3.9	Conclusión	25
4	Regresión logística para clasificación	26
4.1	Objetivos del capítulo	26
4.2	Fundamento matemático	26
4.3	Visualización de la función logística	27
4.4	Cargar paquetes y datos	28
4.5	Preparar y dividir datos	28
4.6	Ajustar modelo	28
4.7	Predicción y matriz de confusión	29
4.8	Métricas	30
4.9	Distribución de probabilidades	30
4.10	Conclusión	31
5	k vecinos más cercanos, k-NN	32
5.1	Objetivos del capítulo	32
5.2	Idea intuitiva	32
5.3	Distancia euclidiana	33
5.4	Cargar y preparar datos	33
5.5	Modelo k-NN	34
5.6	Comparación de valores de k	35
5.7	Conclusión	36
6	Árboles de decisión para clasificación	37
6.1	Objetivos del capítulo	37
6.2	Introducción	37
6.3	Fundamento matemático	37
6.4	Cargar paquetes y datos	37
6.5	Usar la base completa	38
6.6	División y entrenamiento	38
6.7	Complejidad e importancia	39
6.8	Gráfica opcional del árbol	39
6.9	Predicción y métricas	40

6.10	Conclusión	40
7	Random Forest para clasificación	41
7.1	Objetivos del capítulo	41
7.2	Introducción	41
7.3	Fundamento matemático	41
7.4	Cargar paquetes y datos	41
7.5	Muestra de trabajo	42
7.6	División en entrenamiento y prueba	43
7.7	Ajustar Random Forest	44
7.8	Importancia de variables	44
7.9	Predicción y métricas	46
7.10	Distribución de probabilidades predichas	46
7.11	Conclusión	47
8	Evaluación, validación y comparación de modelos	48
8.1	Objetivos del capítulo	48
8.2	Introducción	48
8.3	Cargar los datos	48
8.4	Preparar una muestra reproducible	49
8.5	Entrenamiento, validación y prueba	49
8.6	División estratificada	50
8.7	Matriz de confusión	50
8.8	Métricas principales	51
8.8.1	Exactitud	51
8.8.2	Sensibilidad	51
8.8.3	Especificidad	51
8.8.4	Precisión	51
8.8.5	Valor F1	51
8.9	Función para calcular métricas	52
8.10	Modelo de referencia: regla mayoritaria	53
8.11	Evaluar una regresión logística	54
8.12	Comparación inicial	55
8.13	Umbral de clasificación	56
8.14	Validación cruzada	57
8.14.1	Crear pliegues estratificados	57
8.14.2	Validación cruzada de la regresión logística	58
8.15	Cómo comparar varios modelos	60
8.16	Selección del modelo final	60
8.17	Actividad guiada	61
8.18	Actividades para el lector	61
8.19	Resumen del capítulo	61
9	Máquinas de Vectores de Soporte (SVM)	62
9.1	Objetivos del capítulo	62
9.2	Introducción	62
9.3	Intuición geométrica	62
9.4	Crear un ejemplo didáctico	63
9.5	Visualizar las clases	64

9.6	Instalar y cargar el paquete <code>e1071</code>	65
9.7	Entrenar una SVM lineal	65
9.8	Identificar los vectores de soporte	66
9.9	Dibujar la frontera de decisión	66
9.10	El parámetro de costo	67
9.11	Cuando una línea no es suficiente	68
9.12	Kernel radial	69
10	Aplicación con los datos ATUS	70
10.1	Cargar la base preparada	70
10.2	Preparar una muestra reproducible	70
10.3	Dividir los datos de forma estratificada	71
10.4	Calcular pesos para las clases	72
10.5	Ajustar una SVM lineal	72
10.6	Realizar predicciones	73
10.7	Función segura para calcular métricas	73
10.8	Evaluar la SVM lineal	74
10.9	Ajustar una SVM con kernel radial	74
10.10	Evaluar la SVM radial	75
10.11	Comparar los dos modelos	76
10.12	Selección de hiperparámetros con validación cruzada	77
10.13	Ventajas y limitaciones	78
10.13.1	Ventajas	78
10.13.2	Limitaciones	78
10.14	Recomendaciones prácticas	78
10.15	Actividad guiada	79
10.16	Ejercicios	79
10.17	Conclusiones	79
	Apéndices	81
	Referencias	81
	Reconocimiento de la fuente de datos	81

Presentación

Aprendizaje y Clasificación Automática con R

Un enfoque práctico con datos abiertos reales

Autor: MI Jesús Gilberto Rodríguez Escobedo

Versión: 0.7 (desarrollo)

Modalidad: Libro abierto, práctico e interactivo

Tecnologías: R · Quarto · Machine Learning · Datos abiertos · GitHub Pages

Este libro introduce el aprendizaje automático (*Machine Learning*) mediante ejemplos claros, didácticos y prácticos desarrollados principalmente en **R**.

La intención no es escribir un tratado matemático extenso, sino construir una guía aplicada para aprender haciendo con datos abiertos reales. En esta versión trabajamos con datos de accidentes de tránsito de México y los usamos como hilo conductor para explicar preparación de datos, análisis exploratorio y modelos de clasificación.

Enfoque del libro

El libro combina fundamento conceptual, código reproducible en R, explicación breve del código, interpretación didáctica de resultados y visualizaciones profesionales.

Objetivo general

Desarrollar competencias básicas e intermedias en aprendizaje automático mediante el análisis, modelado e interpretación de datos reales.

Público al que va dirigido

Este libro está dirigido a estudiantes de ingeniería, ciencias, ciencia de datos, estadística aplicada y áreas afines que desean iniciar en aprendizaje automático con un enfoque práctico.

Software utilizado

- RStudio
- Consola de R
- Google Colab
- GitHub
- GitHub Pages
- Quarto Pub

Cuaderno completo de Google Colab

El contenido del libro también está disponible como un cuaderno ejecutable de Google Colab. Las explicaciones se presentan en celdas de texto y cada código R está separado para ejecutarse paso a paso.

- [Abrir el cuaderno en Google Colab](#)
- [Descargar el archivo .ipynb](#)
- [Consultar instrucciones de uso](#)

Estructura actual del libro

1. Introducción al aprendizaje automático.
2. Preparación de datos reales.
3. Análisis exploratorio de datos.
4. Regresión logística.
5. k vecinos más cercanos.
6. Árboles de decisión.
7. Random Forest.
8. Evaluación, validación y comparación de modelos.

Nota para el lector

Los capítulos están diseñados para leerse y ejecutarse paso a paso. Cada bloque de código busca responder una pregunta concreta y cada resultado se acompaña de una interpretación didáctica.

Fuente principal de los datos

Los ejemplos aplicados utilizan información de la Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas del INEGI (Instituto Nacional de Estadística y Geografía 2025). Cuando una tabla o gráfica se elabora con estos datos, se indica expresamente la fuente y se distingue el procesamiento realizado por el autor.

Licencia, autoría y forma de citar

Autor

MI Jesús Gilberto Rodríguez Escobedo

Licencia

Este libro se distribuye bajo la licencia **Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)**.

Esto significa que el material puede compartirse y adaptarse con fines educativos y no comerciales, siempre que se otorgue el crédito correspondiente al autor y las obras derivadas se compartan bajo una licencia compatible.

Cita sugerida

Rodríguez Escobedo, J. G. (2026). *Aprendizaje y Clasificación Automática con R: Un enfoque práctico con datos abiertos reales*. Libro web publicado con Quarto y GitHub Pages.

Uso educativo

El propósito de este libro es apoyar la enseñanza y el aprendizaje de técnicas de aprendizaje automático con R, usando datos reales y un enfoque didáctico, reproducible y aplicado.

Cuaderno completo para Google Colab

Además de la versión web y del documento PDF, el libro cuenta con un **cuaderno completo de Google Colab**. En él, las explicaciones aparecen en celdas de texto y cada bloque de código R se encuentra en una celda independiente para que pueda ejecutarse paso a paso.

 Abrir o descargar el cuaderno

Abrir directamente en Google Colab:

[Ejecutar el libro completo en Google Colab](#)

Descargar el archivo .ipynb:

[Descargar el cuaderno completo](#)

¿Qué contiene el cuaderno?

El cuaderno reproduce el contenido del libro y contiene:

- un índice navegable con todos los capítulos y secciones;
- explicaciones conceptuales en celdas de texto;
- fórmulas matemáticas;
- códigos R separados en celdas independientes;
- instalación y carga de los paquetes necesarios;
- preparación y lectura de los datos ATUS;
- análisis exploratorio;
- regresión logística;
- k vecinos más cercanos;
- árboles de decisión;
- Random Forest;
- evaluación y comparación de modelos;
- ejercicios, interpretaciones y referencias.

Cómo utilizarlo

1. Abra el enlace **Ejecutar el libro completo en Google Colab**.
2. Inicie sesión con una cuenta de Google, si Colab lo solicita.
3. Seleccione **Archivo** → **Guardar una copia en Drive** para conservar una copia editable.
4. Compruebe que el entorno de ejecución utilice **R**.
5. Ejecute primero las celdas de preparación e instalación de paquetes.
6. Continúe en orden, ejecutando cada celda con el botón de reproducción o con `Shift + Enter`.

 Ejecución en orden

Algunos capítulos utilizan objetos creados en capítulos anteriores. Durante la primera ejecución se recomienda avanzar en el orden establecido y no saltar las celdas de preparación de datos.

Diferencias entre el libro web y el cuaderno

La versión web es la opción más cómoda para leer y consultar el contenido. El cuaderno de Colab está pensado para experimentar: permite modificar valores, ejecutar nuevamente los modelos y observar cómo cambian los resultados sin instalar R ni RStudio en la computadora.

 Datos y acceso a internet

El cuaderno necesita conexión a internet para instalar paquetes y descargar los archivos de datos cuando no estén disponibles en la sesión. La descarga de ATUS puede tardar debido al tamaño de la base.

Capítulo 1

¿Qué es el aprendizaje automático?

1.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar qué es el aprendizaje automático, diferenciarlo de la programación tradicional e identificar problemas de clasificación, regresión y agrupamiento.

1.2 Introducción

El aprendizaje automático, también conocido como *Machine Learning*, permite construir modelos capaces de aprender patrones a partir de datos.

En la programación tradicional, una persona define reglas. En aprendizaje automático, proporcionamos ejemplos para que un algoritmo aprenda una relación entre variables de entrada y una salida esperada.

1.3 Programación tradicional contra aprendizaje automático

```
pm10 <- 85

if (pm10 > 75) {
  print("Contaminación alta")
} else {
  print("Contaminación baja")
}
```

```
#> [1] "Contaminación alta"
```

Explicación del código

Se define un valor de PM10 y después se aplica una regla fija. Si el valor es mayor que 75, el programa clasifica el día como de contaminación alta.

Interpretación del resultado

Este ejemplo representa programación tradicional: la decisión depende de una regla escrita manualmente. En aprendizaje automático, la regla se aprende a partir de ejemplos.

```

datos <- data.frame(
  dia = 1:12,
  pm10 = c(42, 88, 55, 120, 63, 95, 38, 110, 72, 130, 47, 99),
  temperatura = c(25, 28, 22, 30, 24, 29, 21, 31, 27, 32, 23, 30),
  humedad = c(40, 35, 60, 30, 55, 33, 65, 29, 42, 28, 62, 34),
  clase = c("Baja", "Alta", "Baja", "Alta", "Baja", "Alta", "Baja", "Alta", "Baja", "Alta",
    ↪ "Baja", "Alta")
)

datos

```

```

#>      dia pm10 temperatura humedad clase
#> 1      1   42           25      40  Baja
#> 2      2   88           28      35  Alta
#> 3      3   55           22      60  Baja
#> 4      4  120           30      30  Alta
#> 5      5   63           24      55  Baja
#> 6      6   95           29      33  Alta
#> 7      7   38           21      65  Baja
#> 8      8  110           31      29  Alta
#> 9      9   72           27      42  Baja
#> 10    10  130           32      28  Alta
#> 11    11   47           23      62  Baja
#> 12    12   99           30      34  Alta

```

Explicación del código

Se construye una pequeña base de datos con 12 días. Cada fila contiene variables ambientales y una clase conocida.

1.4 Variables predictoras y variable respuesta

La variable respuesta es la variable que queremos predecir. Las variables predictoras son las variables que usamos como entrada para el modelo.

1.5 Tipos principales de aprendizaje automático

1. Aprendizaje supervisado.
2. Aprendizaje no supervisado.
3. Aprendizaje por refuerzo.

1.6 Notación básica

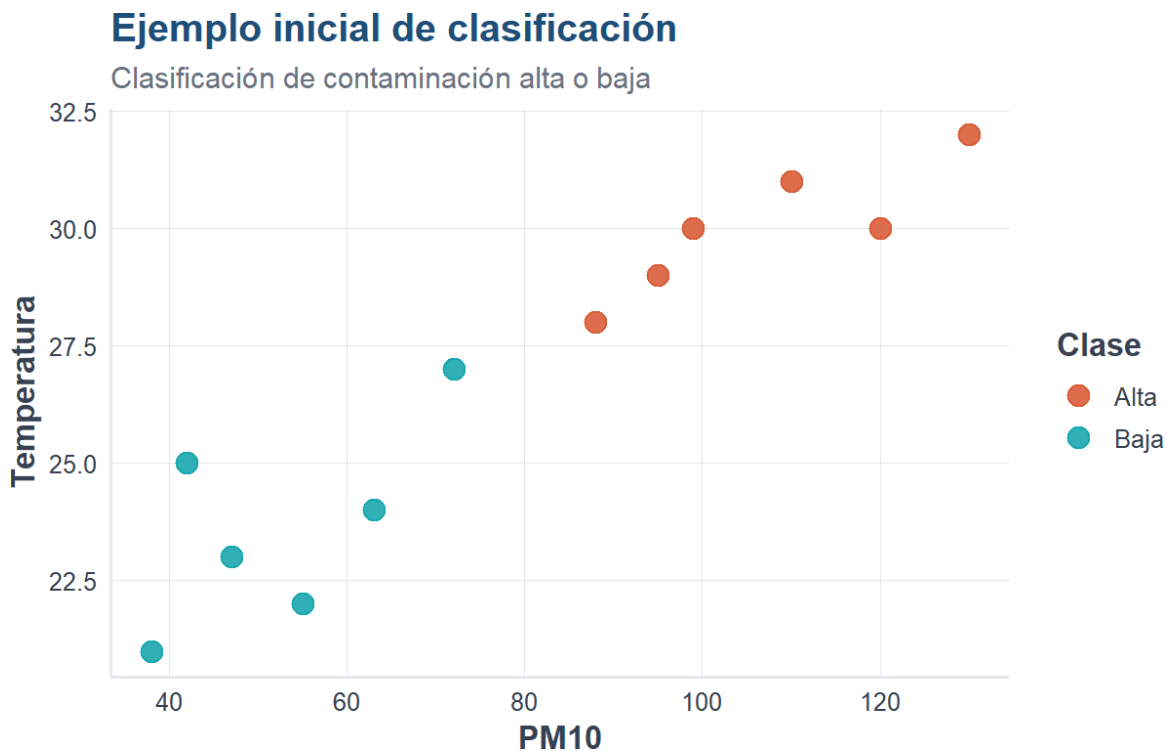
Supongamos que tenemos n observaciones y p variables predictoras. Podemos representar los datos mediante una matriz X y una variable respuesta Y .

$$\hat{y} = f(x_1, x_2, \dots, x_p)$$

1.7 Primer ejemplo visual en R

```
library(ggplot2)
source("util_graficas.R")

ggplot(datos, aes(x = pm10, y = temperatura, color = clase)) +
  geom_point(size = 4, alpha = 0.9) +
  escala_clases_color() +
  labs(
    title = "Ejemplo inicial de clasificación",
    subtitle = "Clasificación de contaminación alta o baja",
    x = "PM10",
    y = "Temperatura",
    color = "Clase"
  ) +
  tema_libro()
```



i Explicación del código

La gráfica coloca PM10 en el eje horizontal, temperatura en el eje vertical y usa color para distinguir la clase.

💡 Interpretación del resultado

Los puntos permiten visualizar si las clases se separan. Esta es la idea básica detrás de muchos métodos de clasificación.

1.8 División entre entrenamiento y prueba

```
set.seed(123)
n <- nrow(datos)
indices_entrenamiento <- sample(1:n, size = round(0.7 * n))

entrenamiento <- datos[indices_entrenamiento, ]
prueba <- datos[-indices_entrenamiento, ]

entrenamiento
```

```
#>   dia pm10 temperatura humedad clase
#> 3    3   55          22      60  Baja
#> 12   12   99          30      34  Alta
#> 10   10  130          32      28  Alta
#> 2    2   88          28      35  Alta
#> 6    6   95          29      33  Alta
#> 11   11   47          23      62  Baja
#> 5    5   63          24      55  Baja
#> 4    4  120          30      30  Alta
```

```
prueba
```

```
#>   dia pm10 temperatura humedad clase
#> 1    1   42          25      40  Baja
#> 7    7   38          21      65  Baja
#> 8    8  110          31      29  Alta
#> 9    9   72          27      42  Baja
```

i Explicación del código

Se divide la base en dos partes: una para entrenar el modelo y otra para evaluar cómo funciona con datos no usados durante el ajuste.

1.9 Primer clasificador basado en una regla

```
datos$prediccion_regla <- ifelse(datos$pm10 > 75, "Alta", "Baja")

tabla_confusion <- table(
  Real = datos$clase,
  Predicho = datos$prediccion_regla
)

tabla_confusion
```

```
#>      Predicho
#> Real  Alta Baja
#> Alta   6    0
#> Baja   0    6
```

```
mean(datos$clase == datos$prediccion_regla)
```

```
#> [1] 1
```

Interpretación del resultado

La matriz de confusión compara la clase real contra la clase predicha. La exactitud mide la proporción de aciertos.

1.10 Conclusión

El aprendizaje automático permite construir modelos que aprenden patrones a partir de datos. En este capítulo vimos la diferencia entre reglas manuales y aprendizaje a partir de ejemplos.

Capítulo 2

Preparación de datos reales

2.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- localizar y descargar la base ATUS desde el portal oficial del INEGI;
- organizar los archivos dentro del proyecto Quarto;
- leer y revisar una base real en R;
- seleccionar y transformar variables;
- construir una variable respuesta binaria;
- guardar una base preparada para los capítulos de modelado;
- citar correctamente la fuente de los datos.

2.2 Introducción

En la práctica profesional, los datos casi nunca llegan preparados para aplicar un algoritmo. Antes de modelar debemos conocer su procedencia, revisar su estructura, limpiar valores, transformar variables y documentar cada decisión.

En este libro utilizamos la **Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS)** del Instituto Nacional de Estadística y Geografía (Instituto Nacional de Estadística y Geografía 2025). Su objetivo es producir información anual sobre la siniestralidad del transporte terrestre en zonas de jurisdicción no federal, con desglose nacional, estatal y municipal.

! Uso responsable de la información

Los datos originales son del INEGI. La limpieza, transformación, selección de variables, gráficas, modelos e interpretaciones de este libro son elaboración del autor. No constituyen resultados oficiales ni implican el aval del Instituto.

2.3 Descargar la base ATUS desde el INEGI

La descarga debe realizarse desde el sitio oficial para conservar la procedencia y los metadatos del archivo.

1. Abra en su navegador el portal de **Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS)**:
<https://www.inegi.org.mx/programas/accidentes/>
2. Localice el apartado **Datos abiertos** o la opción de descarga correspondiente.

3. Seleccione el periodo anual que desea utilizar. Para reproducir los ejemplos de esta versión, elija **2024**.
4. Descargue el archivo en formato CSV. Es posible que el portal entregue un archivo comprimido en formato ZIP.
5. Descomprima el archivo descargado.
6. Identifique el CSV anual y cópielo en la carpeta `datos` del proyecto.
7. Para seguir exactamente los ejemplos, renómbrelo como:

```
atus_2024.csv
```

La estructura mínima del proyecto debe quedar así:

```
libro-machine-learning-r-dev/  
  _quarto.yml  
  index.qmd  
  02-preparacion-datos.qmd  
  datos/  
    atus_2024.csv  
  referencias.bib
```



Otro año de información

Puede utilizar un año diferente. En ese caso, cambie el nombre del archivo o modifique la ruta utilizada en el código. Conviene anotar siempre el año de referencia porque los resultados pueden cambiar entre periodos.

2.4 Cómo citar los datos

Los términos de libre uso del INEGI permiten utilizar, adaptar y publicar su información, pero requieren citar la fuente de origen (Instituto Nacional de Estadística y Geografía 2026).

En el texto puede utilizarse una cita como esta:

Los datos proceden de la Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas del INEGI (Instituto Nacional de Estadística y Geografía 2025).

Debajo de una tabla o gráfica sin transformaciones importantes:

Fuente: INEGI, Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024.

Cuando exista procesamiento, agrupación, modelado o visualización propia:

Fuente: Elaboración propia con datos del INEGI, Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024.

2.5 Cargar paquetes

```
library(readr)
library(dplyr)
library(ggplot2)
library(stringr)
source("util_graficas.R")
```

Explicación del código

Se cargan paquetes para leer archivos CSV, transformar datos, trabajar con texto y crear gráficas. El archivo `util_graficas.R` contiene funciones visuales comunes para mantener un estilo uniforme en todo el libro.

2.6 Localizar el archivo

En lugar de depender únicamente de un nombre fijo, el siguiente código busca archivos cuyo nombre comience con `atus` y termine en `.csv`.

```
archivos_atus <- list.files(
  path = "datos",
  pattern = "^atus.*\\.csv$",
  full.names = TRUE,
  ignore.case = TRUE
)

archivos_atus
```

```
#> [1] "datos/atus_2024.csv"      "datos/atus_ml_preparado.csv"
```

Interpretación

Si aparece al menos una ruta, R encontró un archivo ATUS. Si el resultado es `character(0)`, revise que el archivo esté descomprimido y guardado dentro de la carpeta `datos`.

2.7 Leer el archivo CSV

```
if (length(archivos_atus) == 0) {
  stop(
    paste(
      "No se encontró un archivo ATUS en la carpeta datos.",
      "Descárguelo desde el portal oficial del INEGI y descomprímalo."
    )
  )
}
```

```
ruta_atus <- archivos_atus[1]

atus <- read_csv(
  file = ruta_atus,
  show_col_types = FALSE,
  locale = locale(encoding = "UTF-8")
)

message("Archivo leído: ", ruta_atus)
```

i Explicación del código

Se selecciona el primer archivo localizado y se lee con `read_csv()`. El argumento `show_col_types = FALSE` evita imprimir mensajes técnicos que distraen de los resultados principales.

2.8 Primer vistazo a los datos

```
resumen_dimensiones <- data.frame(
  filas = nrow(atus),
  columnas = ncol(atus)
)

resumen_dimensiones
```

```
#>   filas columnas
#> 1 390627      45
```

```
head(atus)
```

```
#> # A tibble: 6 x 45
#>   COBERTURA ID_ENTIDAD ID_MUNICIPIO ANIO MES ID_HORA ID_MINUTO ID_DIA
#>   <chr>      <chr>      <chr>      <dbl> <chr> <chr>      <chr>      <chr>
#> 1 Municipal 01        001        2024 01    02        25        01
#> 2 Municipal 01        001        2024 01    03        40        01
#> 3 Municipal 01        001        2024 01    04        37        01
#> 4 Municipal 01        001        2024 01    05         0        01
#> 5 Municipal 01        001        2024 01    06        45        01
#> 6 Municipal 01        001        2024 01    07        30        01
#> # i 37 more variables: DIASEMANA <chr>, URBANA <chr>, SUBURBANA <chr>,
#> #   TIPACCID <chr>, AUTOMOVIL <dbl>, CAMPASAJ <dbl>, MICROBUS <dbl>,
#> #   PASCAMION <dbl>, OMNIBUS <dbl>, TRANVIA <dbl>, CAMIONETA <dbl>,
#> #   CAMION <dbl>, TRACTOR <dbl>, FERROCARRI <dbl>, MOTOCICLET <dbl>,
#> #   BICICLETA <dbl>, OTROVEHIC <dbl>, CAUSAACCI <chr>, CAPAROD <chr>,
#> #   SEXO <chr>, ALIENTO <chr>, CINTURON <chr>, ID_EDAD <dbl>, CONDMUERTO <dbl>,
#> #   CONDHERRIDO <dbl>, PASAMUERTO <dbl>, PASAHERIDO <dbl>, PEATMUERTO <dbl>, ...
```

💡 Interpretación

Las filas representan registros de accidentes y las columnas describen sus características temporales, geográficas y operativas. Antes de modelar es indispensable confirmar que las variables esperadas estén disponibles.

2.9 Normalizar los nombres de variables

```
names(atus) <- toupper(names(atus))
names(atus)
```

```
#> [1] "COBERTURA"      "ID_ENTIDAD"      "ID_MUNICIPIO"    "ANIO"            "MES"
#> [6] "ID_HORA"         "ID_MINUTO"       "ID_DIA"          "DIASEMANA"       "URBANA"
#> [11] "SUBURBANA"      "TIPACCID"        "AUTOMOVIL"       "CAMPASAJ"        "MICROBUS"
#> [16] "PASCAMION"      "OMNIBUS"         "TRANVIA"         "CAMIONETA"       "CAMION"
#> [21] "TRACTOR"        "FERROCARRI"      "MOTOCICLET"      "BICICLETA"       "OTROVEHIC"
#> [26] "CAUSAACCI"      "CAPAROD"         "SEXO"            "ALIENTO"         "CINTURON"
#> [31] "ID_EDAD"        "CONDMUERTO"      "CONDHERIDO"      "PASAMUERTO"      "PASAHERIDO"
#> [36] "PEATMUERTO"     "PEATHERIDO"      "CICLMUERTO"      "CICLHERIDO"      "OTROMUERTO"
#> [41] "OTROHERIDO"     "NEMUERTO"        "NEHERIDO"        "CLASACC"         "ESTATUS"
```

Se convierten los nombres a mayúsculas para evitar errores por diferencias como Mes, MES o mes.

2.10 Seleccionar variables de interés

```
variables_interes <- c(
  "ANIO", "MES", "ID_ENTIDAD", "ID_MUNICIPIO",
  "ID_HORA", "ID_DIA", "DIASEMANA", "TIPACCID",
  "CAUSAACCI", "CLASACC"
)

variables_existentes <- intersect(
  variables_interes,
  names(atus)
)

variables_faltantes <- setdiff(
  variables_interes,
  names(atus)
)

atus_base <- atus |>
  select(all_of(variables_existentes))

variables_existentes
```

```
#> [1] "ANIO"      "MES"      "ID_ENTIDAD" "ID_MUNICIPIO" "ID_HORA"
#> [6] "ID_DIA"    "DIASEMANA" "TIPACCID"   "CAUSAACCI"    "CLASACC"
```

```
variables_faltantes
```

```
#> character(0)
```

⚠ Compruebe el diccionario de datos

Los nombres y categorías pueden variar entre versiones. Si aparece una variable faltante, consulte los metadatos y el diccionario de datos descargados junto con la base antes de sustituirla.

2.11 Crear la variable respuesta

El primer problema de clasificación distinguirá entre:

- **Con víctimas:** accidentes fatales o no fatales con personas lesionadas o fallecidas.
- **Solo daños:** accidentes con daños materiales sin víctimas registradas.

```
atus_modelo <- atus_base |>
  mutate(
    CLASACC_TXT = str_to_lower(as.character(CLASACC)),
    accidente_con_victimas = case_when(
      str_detect(CLASACC_TXT, "sólo daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "solo daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "daños") ~ "Solo daños",
      str_detect(CLASACC_TXT, "no fatal") ~ "Con víctimas",
      str_detect(CLASACC_TXT, "fatal") ~ "Con víctimas",
      TRUE ~ NA_character_
    )
  ) |>
  filter(!is.na(accidente_con_victimas))

table(atus_modelo$accidente_con_victimas)
```

```
#>
#> Con víctimas    Solo daños
#>          68108      322519
```

i Explicación del código

La variable CLASACC se transforma en una respuesta binaria. `case_when()` asigna una nueva categoría según el texto encontrado y los registros que no pueden clasificarse se excluyen temporalmente.

2.12 Preparar las variables finales

```

atus_ml <- atus_modelo |>
  mutate(
    MES = as.factor(MES),
    ID_HORA = as.character(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI),
    accidente_con_victimas = factor(
      accidente_con_victimas,
      levels = c("Con víctimas", "Solo daños")
    )
  ) |>
  select(
    accidente_con_victimas,
    MES,
    ID_HORA,
    DIASEMANA,
    TIPACCID,
    CAUSAACCI
  ) |>
  na.omit()

data.frame(
  filas = nrow(atus_ml),
  columnas = ncol(atus_ml)
)

```

```

#>      filas columnas
#> 1 390627          6

```

Interpretación

La base final queda lista para el análisis exploratorio y el modelado. Cada fila representa un accidente y cada columna una variable predictora o la respuesta que se desea clasificar.

2.13 Guardar la base preparada

```

if (!dir.exists("datos")) {
  dir.create("datos")
}

write_csv(
  atus_ml,
  "datos/atus_ml_preparado.csv"
)

```

i Explicación del código

Guardar la base preparada evita repetir toda la limpieza en los capítulos siguientes y ayuda a mantener reproducible el flujo de trabajo.

2.14 Lista de comprobación

Antes de continuar, verifique que:

- el archivo proviene del portal oficial del INEGI;
- el año de los datos está documentado;
- el CSV se encuentra dentro de `datos`;
- las variables usadas existen en la versión descargada;
- la transformación de `CLASACC` corresponde con su diccionario;
- las tablas y gráficas incluyen la fuente;
- el archivo `atus_ml_preparado.csv` se creó correctamente.

2.15 Resumen del capítulo

En este capítulo descargamos y documentamos una base real del INEGI, verificamos su ubicación, seleccionamos variables, construimos una respuesta binaria y guardamos una base preparada para aprendizaje automático.

Capítulo 3

Análisis exploratorio de datos

3.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de cargar una base preparada, revisar su estructura, calcular frecuencias, construir gráficas e interpretar patrones iniciales.

3.2 Cargar paquetes y base

```
library(readr)
library(dplyr)
library(ggplot2)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo 2.")
}
```

```
[1] "Base preparada cargada correctamente."
```

Explicación del código

Se carga la base preparada en el capítulo anterior. Esto permite empezar directamente con el análisis exploratorio.

3.3 Preparar variables

```
if (!is.null(atus_ml)) {
  atus_ml <- atus_ml |>
  mutate(
    ID_HORA_NUM = as.numeric(ID_HORA),
    MES_NUM = as.numeric(MES),
    MES_FACTOR = factor(sprintf("%02d", MES_NUM), levels = sprintf("%02d", 1:12)),
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas", "Solo
    ↪ daños"))
  )
}
```

```
)
}
```

i Explicación del código

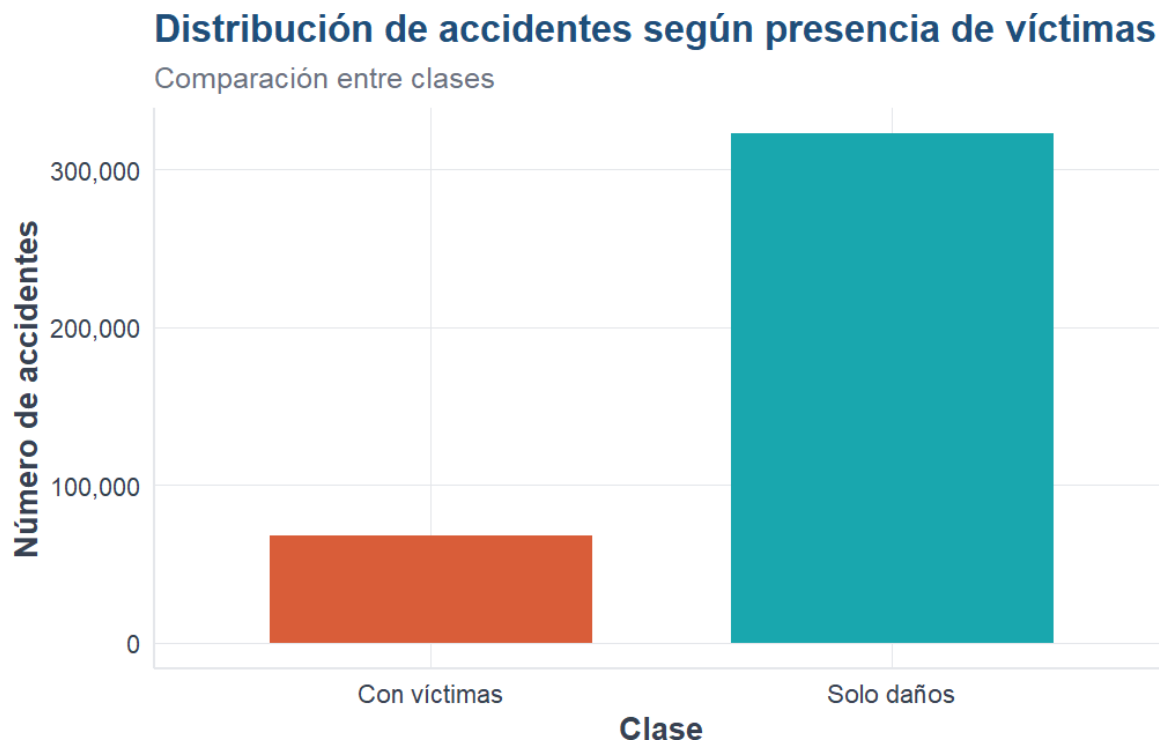
Se crean variables numéricas y factores ordenados para facilitar tablas y gráficas.

3.4 Distribución de la variable respuesta

```
if (!is.null(atus_ml)) {
  tabla_clase <- table(atus_ml$accidente_con_victimas)
  round(100 * prop.table(tabla_clase), 2)
}
```

Con víctimas	Solo daños
17.44	82.56

```
if (!is.null(atus_ml)) {
  ggplot(atus_ml, aes(x = accidente_con_victimas, fill = accidente_con_victimas)) +
    geom_bar(width = 0.7) +
    escala_clases_fill() +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Distribución de accidentes según presencia de víctimas",
      subtitle = "Comparación entre clases",
      x = "Clase",
      y = "Número de accidentes",
      fill = "Clase"
    ) +
    tema_libro() +
    theme(legend.position = "none")
}
```



💡 Interpretación del resultado

La base está desbalanceada: hay más accidentes de solo daños que accidentes con víctimas. Esto será importante al evaluar modelos.

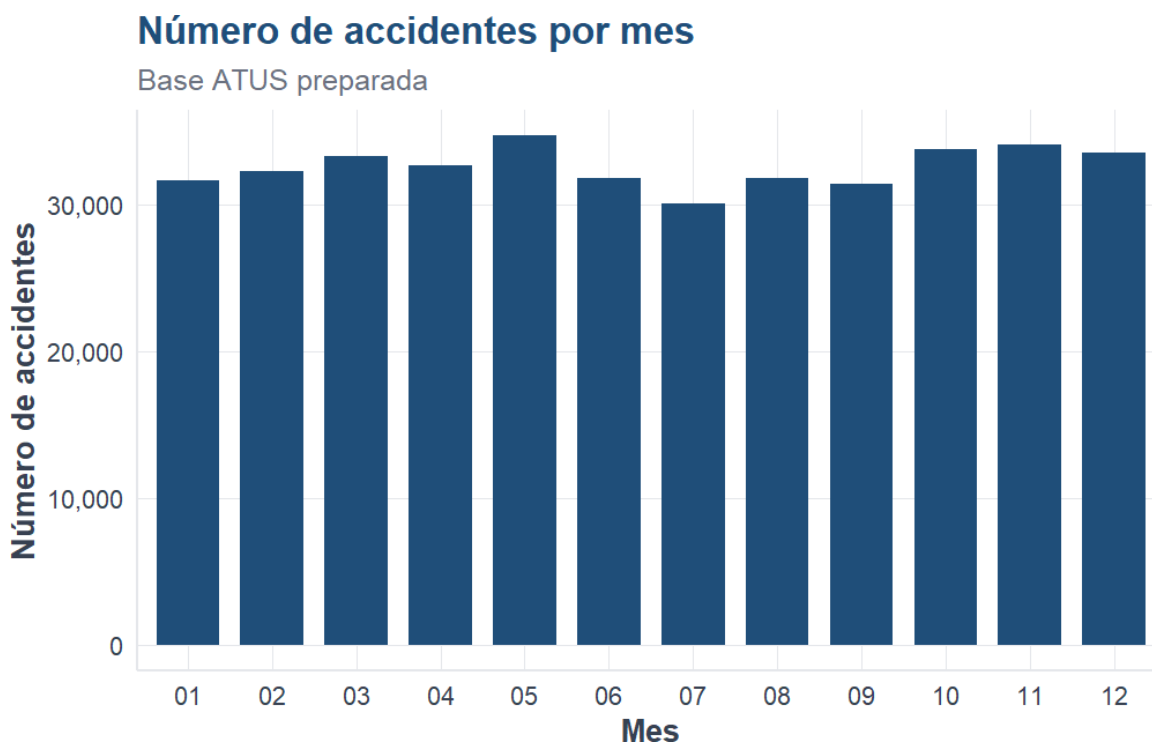
3.5 Accidentes por mes

```
if (!is.null(atus_ml)) {
  accidentes_mes <- atus_ml |> count(MES_FACTOR, name = "n") |> arrange(MES_FACTOR)
  accidentes_mes
}
```

```
# A tibble: 12 x 2
  MES_FACTOR      n
  <fct>         <int>
1 01           31600
2 02           32208
3 03           33237
4 04           32666
5 05           34665
6 06           31737
7 07           30062
8 08           31800
9 09           31351
10 10          33771
```

```
11 11          34047
12 12          33483
```

```
if (exists("accidentes_mes")) {
  ggplot(accidentes_mes, aes(x = MES_FACTOR, y = n)) +
    geom_col(fill = col_azul, width = 0.75) +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Número de accidentes por mes",
      subtitle = "Base ATUS preparada",
      x = "Mes",
      y = "Número de accidentes"
    ) +
    tema_libro()
}
```



i Explicación del código

Se cuentan los accidentes por mes y se visualizan con barras. El eje vertical usa separadores de miles para facilitar la lectura.

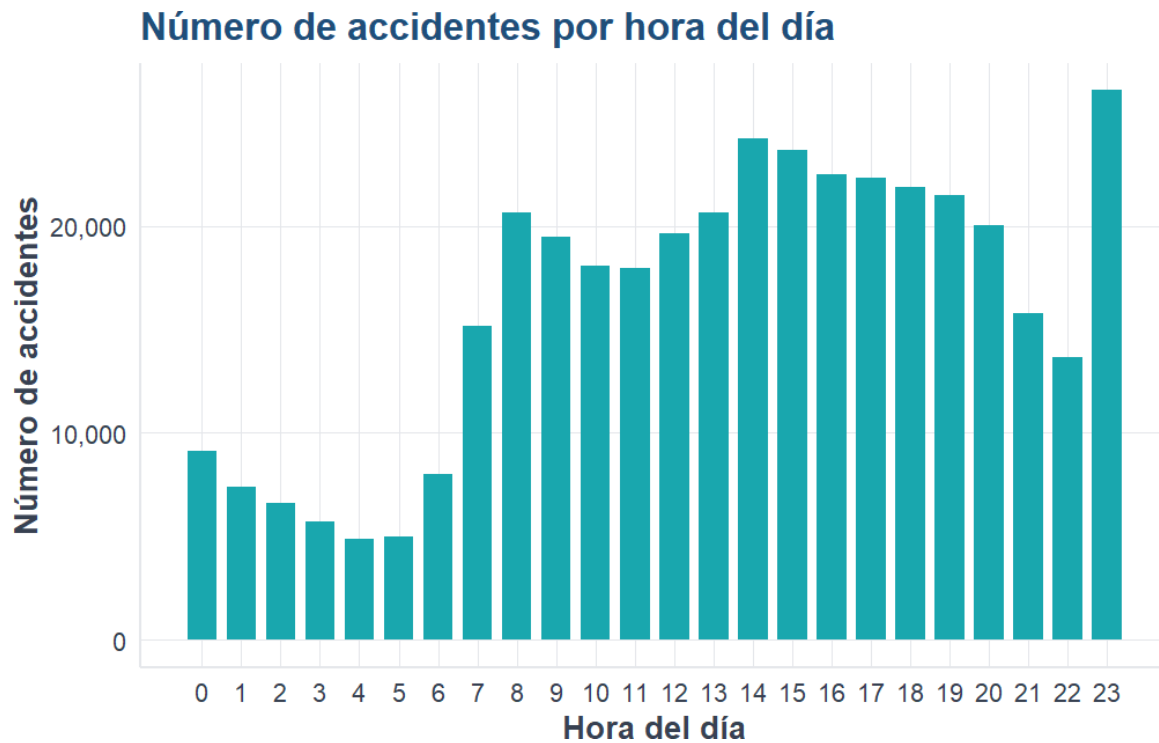
3.6 Accidentes por hora del día

```
if (!is.null(atus_ml)) {
  accidentes_hora <- atus_ml |> count(ID_HORA_NUM, name = "n") |> arrange(ID_HORA_NUM)

  ggplot(accidentes_hora, aes(x = ID_HORA_NUM, y = n)) +
    geom_col(fill = col_turquesa, width = 0.75) +

```

```
scale_x_continuous(breaks = 0:23) +
scale_y_continuous(labels = etiqueta_numero) +
labs(
  title = "Número de accidentes por hora del día",
  x = "Hora del día",
  y = "Número de accidentes"
) +
tema_libro()
}
```



💡 Interpretación del resultado

La gráfica permite detectar horas con mayor concentración de accidentes.

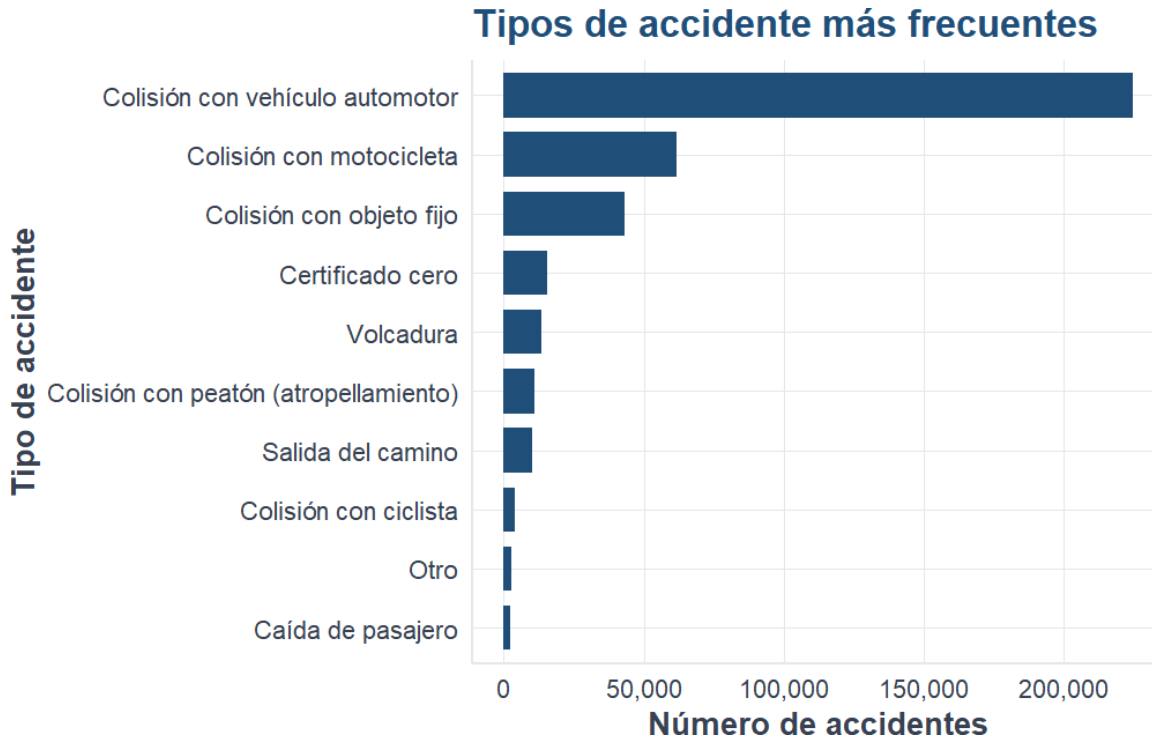
3.7 Tipos de accidente más frecuentes

```
if (!is.null(atus_ml)) {
  accidentes_tipo_top <- atus_ml |> count(TIPACCID, name = "n") |> slice_max(n, n = 10)

  ggplot(accidentes_tipo_top, aes(x = reorder(TIPACCID, n), y = n)) +
    geom_col(fill = col_azul, width = 0.75) +
    coord_flip() +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Tipos de accidente más frecuentes",
      x = "Tipo de accidente",
      y = "Número de accidentes"
    ) +
    tema_libro()
}
```

```
}

```



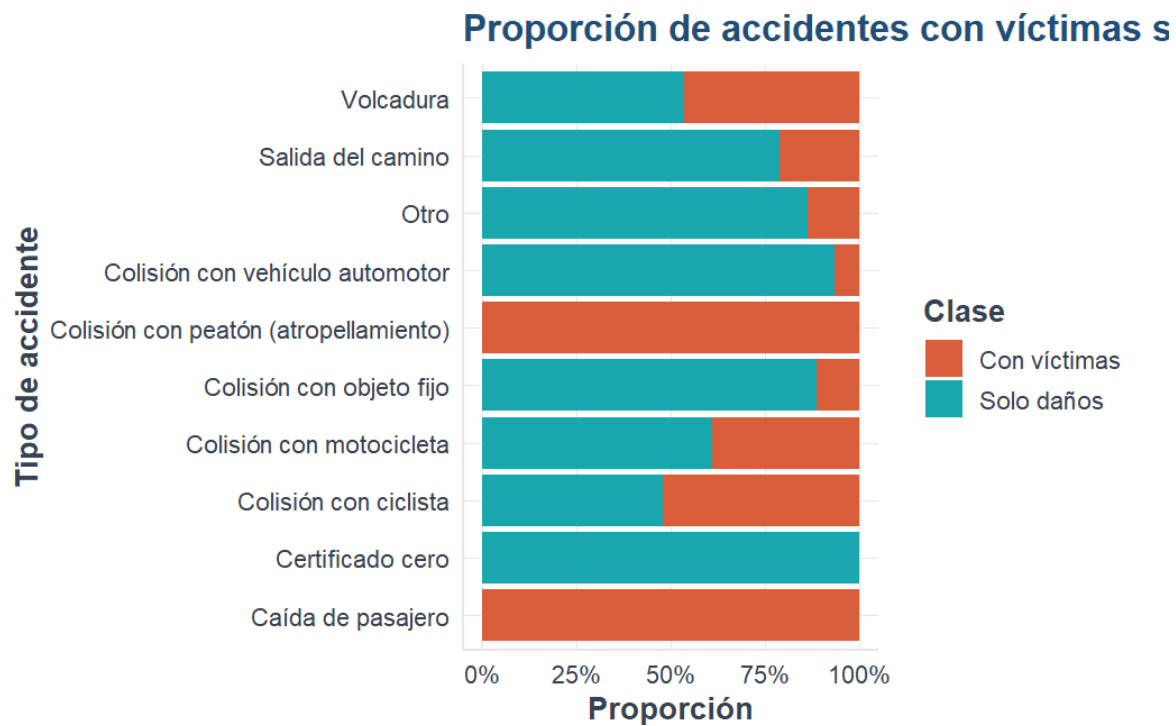
i Explicación del código

Se seleccionan los diez tipos de accidente con mayor frecuencia para evitar una gráfica saturada.

3.8 Proporción por tipo de accidente

```
if (!is.null(atus_ml)) {
  tipos_top <- atus_ml |> count(TIPACCID) |> slice_max(n, n = 10) |> pull(TIPACCID)
  atus_tipo_top <- atus_ml |> filter(TIPACCID %in% tipos_top)

  ggplot(atus_tipo_top, aes(x = TIPACCID, fill = accidente_con_victimas)) +
    geom_bar(position = "fill") +
    coord_flip() +
    escala_clases_fill() +
    scale_y_continuous(labels = etiqueta_porcentaje) +
    labs(
      title = "Proporción de accidentes con víctimas según tipo de accidente",
      x = "Tipo de accidente",
      y = "Proporción",
      fill = "Clase"
    ) +
    tema_libro()
}
```



💡 Interpretación del resultado

Esta gráfica compara proporciones, no cantidades absolutas. Permite identificar tipos de accidente con mayor presencia relativa de víctimas.

3.9 Conclusión

El análisis exploratorio permite detectar patrones iniciales, revisar el desbalance de clases e identificar variables útiles para modelar.

Capítulo 4

Regresión logística para clasificación

4.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de ajustar una regresión logística, predecir probabilidades y evaluar el desempeño.

4.2 Fundamento matemático

La regresión logística utiliza la función:

$$p = \frac{1}{1 + e^{-z}}$$

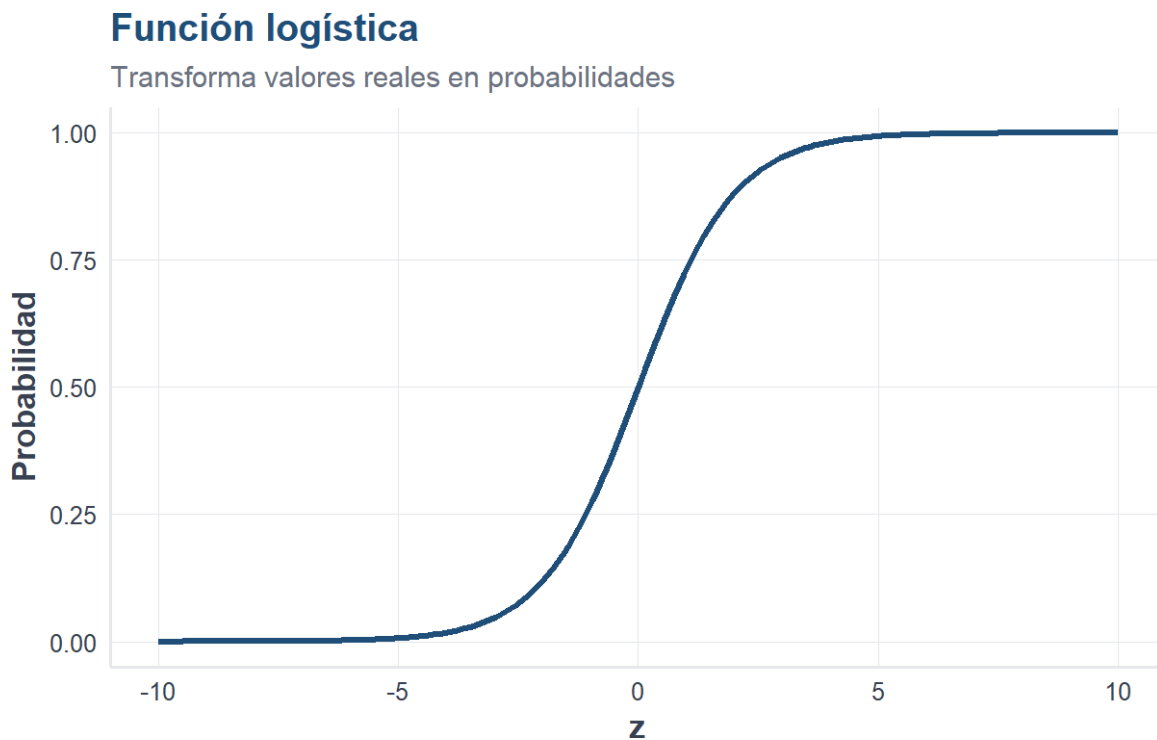
donde $z = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$.

4.3 Visualización de la función logística

```
library(ggplot2)
source("util_graficas.R")

z <- seq(-10, 10, length.out = 200)
datos_logistica <- data.frame(z = z, p = 1 / (1 + exp(-z)))

ggplot(datos_logistica, aes(x = z, y = p)) +
  geom_line(linewidth = 1.2, color = col_azul) +
  labs(
    title = "Función logística",
    subtitle = "Transforma valores reales en probabilidades",
    x = "z",
    y = "Probabilidad"
  ) +
  tema_libro()
```



Explicación del código

Se genera una secuencia de valores para z y se calcula su probabilidad usando la función logística.

Interpretación del resultado

La curva tiene forma de S. Valores negativos producen probabilidades cercanas a cero y valores positivos probabilidades cercanas a uno.

4.4 Cargar paquetes y datos

```
library(readr)
library(dplyr)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo 2.")
}
```

```
#> [1] "Base preparada cargada correctamente."
```

4.5 Preparar y dividir datos

```
if (!is.null(atus_ml)) {
  atus_modelo <- atus_ml |>
  mutate(
    victimas_binaria = ifelse(accidente_con_victimas == "Con víctimas", 1, 0),
    MES = as.factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI),
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas", "Solo
    ↪ daños"))
  )

  set.seed(123)
  idx <- sample(1:nrow(atus_modelo), size = round(0.7 * nrow(atus_modelo)))
  entrenamiento <- atus_modelo[idx, ]
  prueba <- atus_modelo[-idx, ]
}
```

i Explicación del código

La variable respuesta se transforma a binaria y la base se divide en entrenamiento y prueba.

4.6 Ajustar modelo

```
if (exists("entrenamiento")) {
  modelo_logistico <- glm(
    victimas_binaria ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
```

```

    data = entrenamiento,
    family = binomial
  )

  head(summary(modelo_logistico)$coefficients, 12)
}

```

```

#>               Estimate Std. Error   z value   Pr(>|z|)
#> (Intercept) 18.107257168 101.84719297  0.17778848 8.588891e-01
#> MES02      -0.045765700  0.02975556 -1.53805548 1.240350e-01
#> MES03      -0.032191364  0.02927313 -1.09968980 2.714673e-01
#> MES04      -0.002838049  0.02945462 -0.09635327 9.232400e-01
#> MES05      -0.015408552  0.02906672 -0.53010979 5.960358e-01
#> MES06      -0.054753129  0.02983667 -1.83509526 6.649158e-02
#> MES07      -0.076296781  0.03035194 -2.51373620 1.194598e-02
#> MES08      -0.094049541  0.02997105 -3.13801278 1.700975e-03
#> MES09      -0.077109438  0.03015044 -2.55748998 1.054306e-02
#> MES10      -0.154152374  0.02979697 -5.17342485 2.298416e-07
#> MES11      -0.106385033  0.02950811 -3.60528073 3.118157e-04
#> MES12      -0.111888509  0.02959338 -3.78086333 1.562855e-04

```

Interpretación del resultado

Los coeficientes estiman el efecto de cada variable sobre el logit de la probabilidad de accidente con víctimas.

4.7 Predicción y matriz de confusión

```

if (exists("modelo_logistico")) {
  prueba$probabilidad_victimas <- predict(modelo_logistico, newdata = prueba, type = "response")
  prueba$prediccion_03 <- factor(
    ifelse(prueba$probabilidad_victimas >= 0.3, "Con víctimas", "Solo daños"),
    levels = c("Con víctimas", "Solo daños")
  )

  matriz_confusion_03 <- table(
    Real = prueba$saccidente_con_victimas,
    Predicho = prueba$prediccion_03
  )

  matriz_confusion_03
}

```

```

#>               Predicho
#> Real              Con víctimas Solo daños
#> Con víctimas      13765      6739
#> Solo daños       14302      82382

```

Explicación del código

Las probabilidades se convierten en clases usando un punto de corte de 0.3.

4.8 Métricas

```
if (exists("matriz_confusion_03")) {
  VP <- matriz_confusion_03["Con víctimas", "Con víctimas"]
  FN <- matriz_confusion_03["Con víctimas", "Solo daños"]
  FP <- matriz_confusion_03["Solo daños", "Con víctimas"]
  VN <- matriz_confusion_03["Solo daños", "Solo daños"]

  data.frame(
    exactitud = (VP + VN) / (VP + FN + FP + VN),
    sensibilidad = VP / (VP + FN),
    especificidad = VN / (VN + FP)
  )
}
```

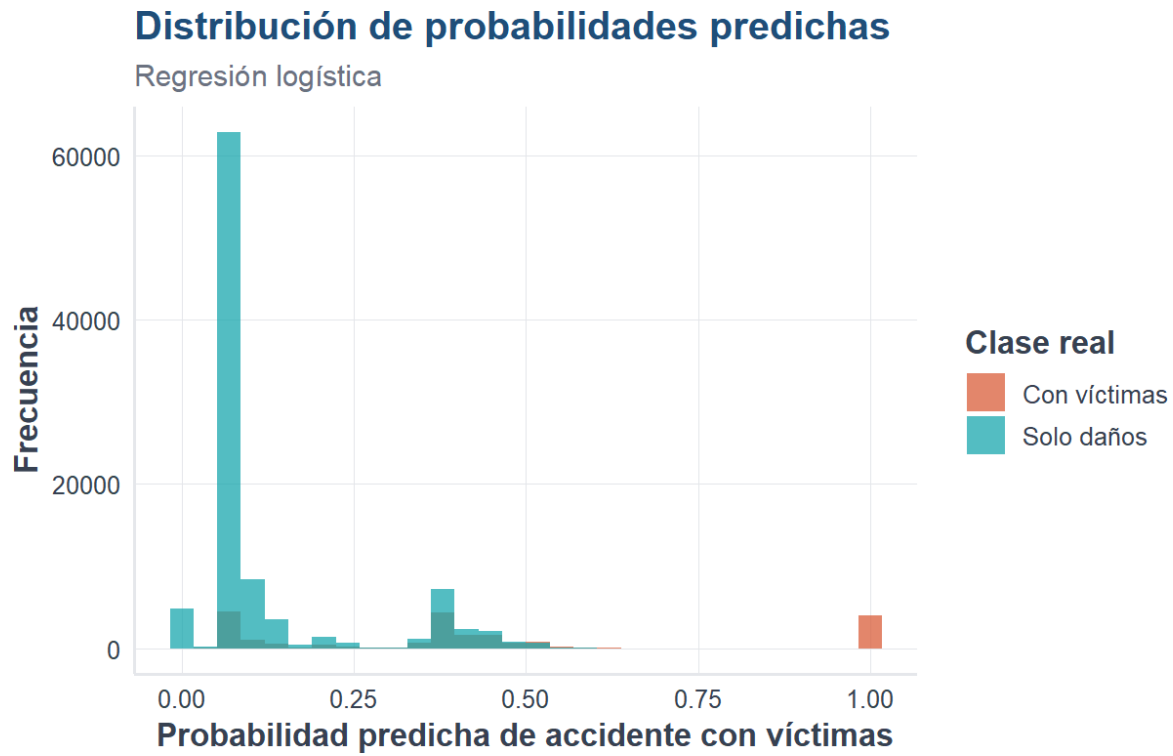
```
#> exactitud sensibilidad especificidad
#> 1 0.8204509 0.6713324 0.8520748
```

Interpretación del resultado

La sensibilidad mide la capacidad para detectar accidentes con víctimas; la especificidad mide la capacidad para detectar accidentes de solo daños.

4.9 Distribución de probabilidades

```
if (exists("prueba")) {
  ggplot(prueba, aes(x = probabilidad_victimas, fill = accidente_con_victimas)) +
    geom_histogram(bins = 30, alpha = 0.75, position = "identity") +
    escala_clases_fill(name = "Clase real") +
    labs(
      title = "Distribución de probabilidades predichas",
      subtitle = "Regresión logística",
      x = "Probabilidad predicha de accidente con víctimas",
      y = "Frecuencia"
    ) +
    tema_libro()
}
```



4.10 Conclusión

La regresión logística es un modelo interpretable para clasificación binaria. El punto de corte permite ajustar el balance entre sensibilidad y especificidad.

Capítulo 5

k vecinos más cercanos, k-NN

5.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar k-NN, preparar datos para este método, estandarizar variables y evaluar el desempeño.

5.2 Idea intuitiva

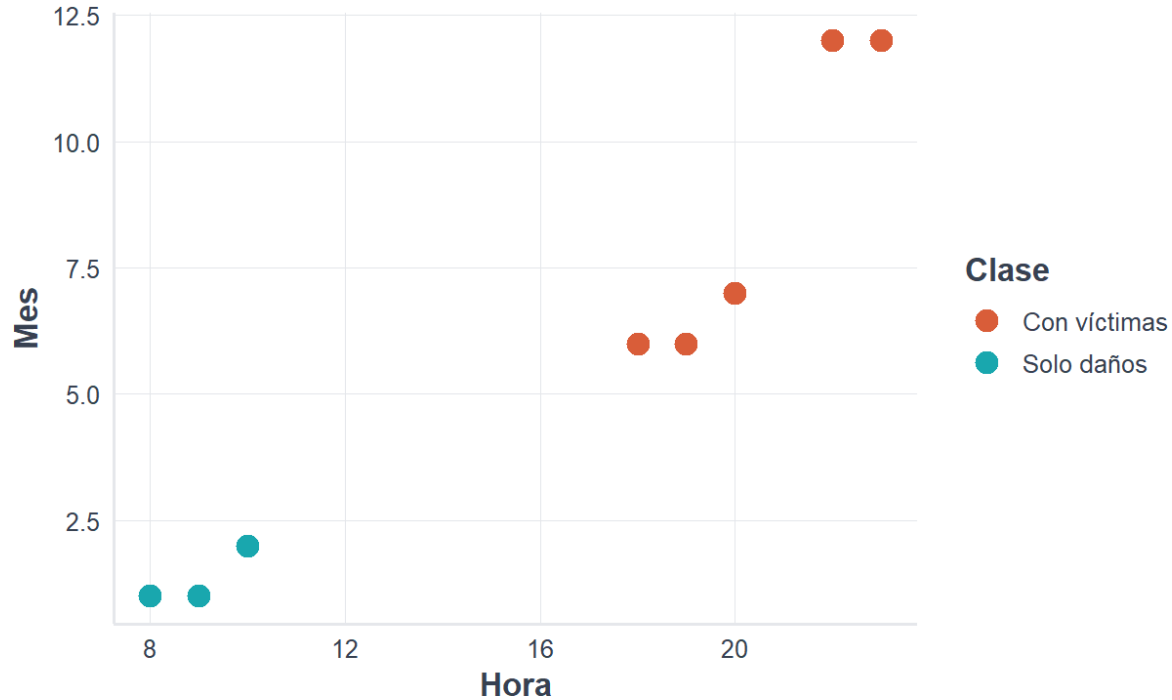
k-NN clasifica una nueva observación según la clase más común entre sus vecinos más cercanos.

```
library(ggplot2)
library(class)
library(dplyr)
library(tidyr)
library(readr)
source("util_graficas.R")

datos_ejemplo <- data.frame(
  hora = c(8, 9, 10, 18, 19, 20, 22, 23),
  mes = c(1, 1, 2, 6, 6, 7, 12, 12),
  clase = c("Solo daños", "Solo daños", "Solo daños", "Con víctimas", "Con víctimas", "Con
    ↪ víctimas", "Con víctimas", "Con víctimas")
)

ggplot(datos_ejemplo, aes(x = hora, y = mes, color = clase)) +
  geom_point(size = 4) +
  escala_clases_color() +
  labs(title = "Ejemplo intuitivo de k-NN", x = "Hora", y = "Mes", color = "Clase") +
  tema_libro()
```

Ejemplo intuitivo de k-NN



Explicación del código

Se construyen puntos artificiales para visualizar la idea de vecinos cercanos.

5.3 Distancia euclidiana

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

```
sqrt((8 - 10)^2 + (1 - 2)^2)
```

```
#> [1] 2.236068
```

Interpretación del resultado

Una distancia menor indica mayor similitud entre dos observaciones.

5.4 Cargar y preparar datos

```
ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
```

```
set.seed(123)
atus_knn_base <- atus_ml |>
  sample_n(min(12000, nrow(atus_ml))) |>
  mutate(
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas", "Solo
    ↪ daños")),
    MES = as.factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI)
  ) |>
  na.omit()
```

i Explicación del código

Se toma una muestra para que k-NN sea rápido y reproducible. Este método puede ser costoso en bases grandes.

5.5 Modelo k-NN

```
if (exists("atus_knn_base")) {
  matriz_predictoras <- model.matrix(~ ID_HORA + MES + DIASEMANA + TIPACCID + CAUSAACCI - 1,
  ↪ data = atus_knn_base)
  y <- atus_knn_base$accidente_con_victimas

  set.seed(123)
  idx <- sample(1:nrow(matriz_predictoras), size = round(0.7 * nrow(matriz_predictoras)))
  x_entrenamiento <- matriz_predictoras[idx, ]
  x_prueba <- matriz_predictoras[-idx, ]
  y_entrenamiento <- y[idx]
  y_prueba <- y[-idx]

  medias <- apply(x_entrenamiento, 2, mean)
  desviaciones <- apply(x_entrenamiento, 2, sd)
  desviaciones[desviaciones == 0] <- 1

  x_entrenamiento_esc <- scale(x_entrenamiento, center = medias, scale = desviaciones)
  x_prueba_esc <- scale(x_prueba, center = medias, scale = desviaciones)

  pred_knn_5 <- knn(x_entrenamiento_esc, x_prueba_esc, y_entrenamiento, k = 5)
  table(Real = y_prueba, Predicho = pred_knn_5)
}
```

```
#>               Predicho
#> Real           Con víctimas Solo daños
#> Con víctimas      211         399
#> Solo daños       144        2846
```


i Explicación del código

Se crean variables dummy, se estandarizan las columnas y se clasifica con k igual a 5.

5.6 Comparación de valores de k

```
calcular_metricas <- function(real, predicho) {
  real <- factor(real, levels = c("Con víctimas", "Solo daños"))
  predicho <- factor(predicho, levels = c("Con víctimas", "Solo daños"))
  m <- table(Real = real, Predicho = predicho)
  VP <- m["Con víctimas", "Con víctimas"]
  FN <- m["Con víctimas", "Solo daños"]
  FP <- m["Solo daños", "Con víctimas"]
  VN <- m["Solo daños", "Solo daños"]
  data.frame(exactitud=(VP+VN)/(VP+FN+FP+VN), sensibilidad=VP/(VP+FN), especificidad=VN/(VN+FP))
}

if (exists("x_entrenamiento_esc")) {
  valores_k <- c(3, 5, 11)
  resultados_knn <- data.frame()

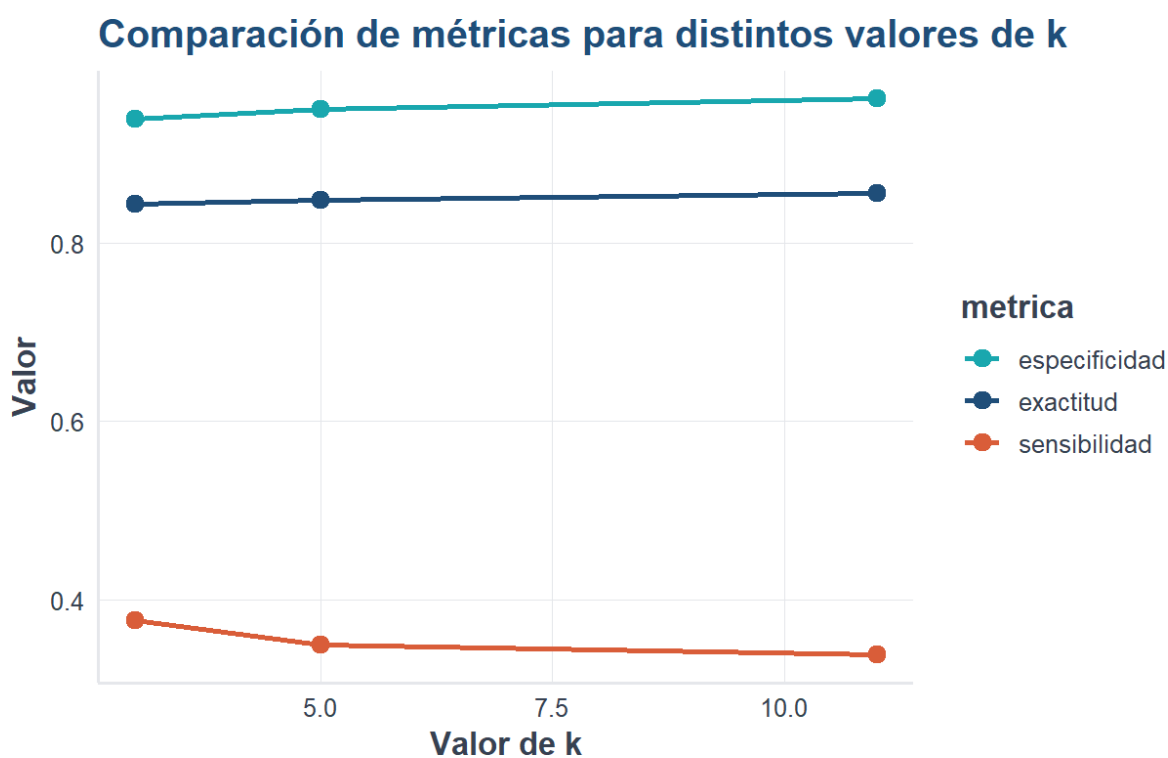
  for (k_actual in valores_k) {
    pred_actual <- knn(x_entrenamiento_esc, x_prueba_esc, y_entrenamiento, k = k_actual)
    tmp <- calcular_metricas(y_prueba, pred_actual)
    tmp$k <- k_actual
    resultados_knn <- rbind(resultados_knn, tmp)
  }

  resultados_knn <- resultados_knn |> select(k, exactitud, sensibilidad, especificidad)
  resultados_knn
}
```

```
#>      k exactitud sensibilidad especificidad
#> 1   3 0.8433333    0.3770492    0.9384615
#> 2   5 0.8480556    0.3491803    0.9498328
#> 3  11 0.8558333    0.3377049    0.9615385
```

```
if (exists("resultados_knn")) {
  resultados_largos <- resultados_knn |>
    pivot_longer(cols = c(exactitud, sensibilidad, especificidad), names_to = "metrica",
    ↪ values_to = "valor")

  ggplot(resultados_largos, aes(x = k, y = valor, color = metrica, group = metrica)) +
    geom_line(linewidth = 1.1) +
    geom_point(size = 3) +
    scale_color_manual(values = c(exactitud = col_azul, sensibilidad = col_coral, especificidad
    ↪ = col_turquesa)) +
    labs(title = "Comparación de métricas para distintos valores de k", x = "Valor de k", y =
    ↪ "Valor") +
    tema_libro()
}
```



💡 Interpretación del resultado

Cambiar k modifica el balance entre exactitud, sensibilidad y especificidad. No existe un k universalmente mejor.

5.7 Conclusión

k-NN es un método intuitivo basado en similitud. Su desempeño depende del valor de k , del escalamiento y de la calidad de las variables predictoras.

Capítulo 6

Árboles de decisión para clasificación

6.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar la idea básica de un árbol de decisión, entrenarlo en R y evaluar su desempeño.

6.2 Introducción

Los árboles de decisión clasifican observaciones mediante una secuencia de preguntas. Son interpretables y pueden expresarse como reglas tipo si-entonces.

6.3 Fundamento matemático

Una medida común de impureza es el índice de Gini:

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

6.4 Cargar paquetes y datos

```
library(readr)
library(dplyr)
library(rpart)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo 2.")
}
```

```
[1] "Base preparada cargada correctamente."
```

6.5 Usar la base completa

```
if (!is.null(atus_ml)) {
  atus_arbol_base <- atus_ml |>
  mutate(
    accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas", "Solo
    ↪ daños")),
    MES = as.factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = as.factor(DIASEMANA),
    TIPACCID = as.factor(TIPACCID),
    CAUSAACCI = as.factor(CAUSAACCI)
  ) |>
  na.omit()

  data.frame(filas = nrow(atus_arbol_base), columnas = ncol(atus_arbol_base))
}
```

```
      filas columnas
1 390627           6
```

i Explicación del código

Se usa la base completa, pero el crecimiento del árbol se controla mediante parámetros del modelo.

6.6 División y entrenamiento

```
if (exists("atus_arbol_base")) {
  set.seed(123)
  idx <- sample(1:nrow(atus_arbol_base), size = round(0.7 * nrow(atus_arbol_base)))
  entrenamiento <- atus_arbol_base[idx, ]
  prueba <- atus_arbol_base[-idx, ]

  modelo_arbol <- rpart(
    accidente_con_victimas ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
    data = entrenamiento,
    method = "class",
    control = rpart.control(cp = 0.005, minsplit = 1000, minbucket = 300, maxdepth = 5)
  )
}
```

i Explicación del código

Se ajusta un árbol de decisión controlado para evitar sobreajuste y salidas demasiado grandes.

6.7 Complejidad e importancia

```
if (exists("modelo_arbol")) printcp(modelo_arbol)
```

Classification tree:

```
rpart(formula = accidente_con_victimas ~ MES + ID_HORA + DIASEMANA +
      TIPACCID + CAUSAACCI, data = entrenamiento, method = "class",
      control = rpart.control(cp = 0.005, minsplit = 1000, minbucket = 300,
                              maxdepth = 5))
```

Variables actually used in tree construction:

```
[1] TIPACCID
```

Root node error: 47604/273439 = 0.17409

n= 273439

	CP	nsplit	rel error	xerror	xstd
1	0.097051	0	1.0000	1.0000	0.0041653
2	0.005000	2	0.8059	0.8059	0.0038150

```
if (exists("modelo_arbol")) {
  importancia <- modelo_arbol$variable.importance
  if (!is.null(importancia)) {
    importancia_df <- data.frame(variable = names(importancia), importancia =
  ↪ as.numeric(importancia), row.names = NULL)
    importancia_df <- importancia_df[order(-importancia_df$importancia), ]
    importancia_df
  }
}
```

	variable	importancia
1	TIPACCID	22919.330
2	CAUSAACCI	1093.984

Interpretación del resultado

La importancia de variables indica qué variables ayudan más a separar accidentes con víctimas de accidentes de solo daños.

6.8 Gráfica opcional del árbol

La gráfica completa del árbol puede ser pesada en PDF. El lector puede ejecutarla manualmente en RStudio.

```
plot(modelo_arbol, uniform = TRUE, margin = 0.1)
text(modelo_arbol, use.n = TRUE, cex = 0.7)
```

6.9 Predicción y métricas

```
if (exists("modelo_arbol")) {
  pred_arbol <- predict(modelo_arbol, newdata = prueba, type = "class")
  real_arbol <- factor(prueba$accidente_con_victimas, levels = c("Con víctimas", "Solo daños"))
  pred_arbol <- factor(pred_arbol, levels = c("Con víctimas", "Solo daños"))

  matriz_confusion_arbol <- table(Real = real_arbol, Predicho = pred_arbol)
  matriz_confusion_arbol
}
```

	Predicho	
Real	Con víctimas	Solo daños
Con víctimas	3965	16539
Solo daños	0	96684

```
if (exists("matriz_confusion_arbol")) {
  VP <- matriz_confusion_arbol["Con víctimas", "Con víctimas"]
  FN <- matriz_confusion_arbol["Con víctimas", "Solo daños"]
  FP <- matriz_confusion_arbol["Solo daños", "Con víctimas"]
  VN <- matriz_confusion_arbol["Solo daños", "Solo daños"]

  data.frame(exactitud=(VP+VN) / (VP+FN+FP+VN), sensibilidad=VP / (VP+FN), especificidad=VN / (VN+FP))
}
```

	exactitud	sensibilidad	especificidad
1	0.8588678	0.1933769	1

Interpretación del resultado

La matriz de confusión y las métricas permiten evaluar el desempeño del árbol sobre datos de prueba.

6.10 Conclusión

Los árboles de decisión son modelos intuitivos, visuales e interpretables. Este capítulo prepara el camino para modelos basados en muchos árboles, como Random Forest.

Capítulo 7

Random Forest para clasificación

7.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de explicar Random Forest, entrenar un modelo en R, interpretar importancia de variables y evaluar desempeño.

7.2 Introducción

Random Forest combina muchos árboles de decisión. Cada árbol aprende sobre una muestra aleatoria de los datos y la predicción final se obtiene por votación mayoritaria.

7.3 Fundamento matemático

Si se construyen B árboles, la predicción final es:

$$\hat{y}(x) = \text{moda}\{\hat{y}^{(1)}(x), \hat{y}^{(2)}(x), \dots, \hat{y}^{(B)}(x)\}$$

7.4 Cargar paquetes y datos

```
library(readr)
library(dplyr)
library(ggplot2)
library(ranger)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (file.exists(ruta_atus_ml)) {
  atus_ml <- read_csv(ruta_atus_ml, show_col_types = FALSE)
  print("Base preparada cargada correctamente.")
} else {
  atus_ml <- NULL
  print("No se encontró el archivo datos/atus_ml_preparado.csv. Ejecuta primero el capítulo 2.")
}
```

```
#> [1] "Base preparada cargada correctamente."
```

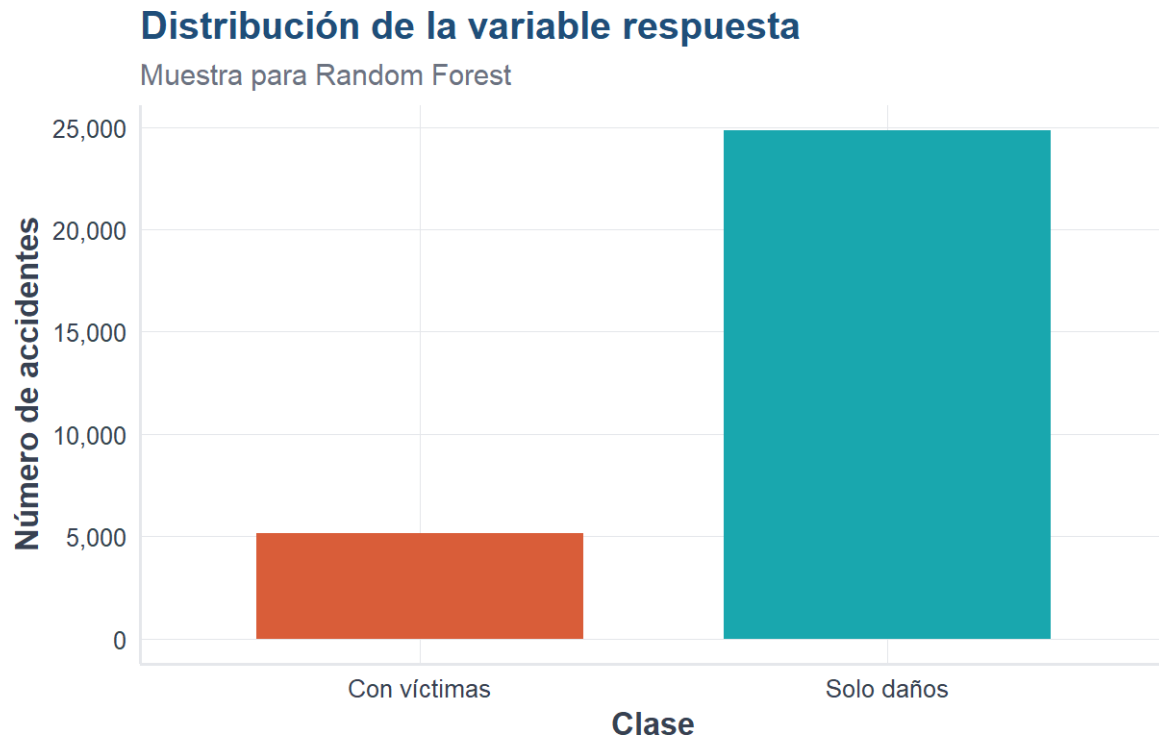
7.5 Muestra de trabajo

```
if (!is.null(atus_ml)) {
  set.seed(123)
  atus_rf_base <- atus_ml |>
    sample_n(min(30000, nrow(atus_ml))) |>
    mutate(
      accidente_con_victimas = factor(accidente_con_victimas, levels = c("Con víctimas", "Solo
        ↪ daños")),
      MES = as.factor(MES),
      ID_HORA = as.numeric(ID_HORA),
      DIASEMANA = as.factor(DIASEMANA),
      TIPACCID = as.factor(TIPACCID),
      CAUSAACCI = as.factor(CAUSAACCI)
    ) |>
    na.omit()

  data.frame(filas = nrow(atus_rf_base), columnas = ncol(atus_rf_base))
}
```

```
#>   filas columnas
#> 1 30000         6
```

```
if (exists("atus_rf_base")) {
  ggplot(atus_rf_base, aes(x = accidente_con_victimas, fill = accidente_con_victimas)) +
    geom_bar(width = 0.7) +
    escala_clases_fill() +
    scale_y_continuous(labels = etiqueta_numero) +
    labs(
      title = "Distribución de la variable respuesta",
      subtitle = "Muestra para Random Forest",
      x = "Clase",
      y = "Número de accidentes"
    ) +
    tema_libro() +
    theme(legend.position = "none")
}
```

i Explicación del código

Se toma una muestra de trabajo para mantener el tiempo de ejecución razonable. El modelo Random Forest suele ser más pesado que un solo árbol.

7.6 División en entrenamiento y prueba

```
if (exists("atus_rf_base")) {
  set.seed(123)
  idx <- sample(1:nrow(atus_rf_base), size = round(0.7 * nrow(atus_rf_base)))
  entrenamiento <- atus_rf_base[idx, ]
  prueba <- atus_rf_base[-idx, ]

  data.frame(conjunto = c("Entrenamiento", "Prueba"), registros = c(nrow(entrenamiento),
    ↪ nrow(prueba)))
}
```

```
#>      conjunto registros
#> 1 Entrenamiento    21000
#> 2      Prueba      9000
```

7.7 Ajustar Random Forest

```
if (exists("entrenamiento")) {
  set.seed(123)

  modelo_rf <- ranger(
    accidente_con_victimas ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
    data = entrenamiento,
    num.trees = 200,
    mtry = 3,
    min.node.size = 20,
    importance = "impurity",
    probability = TRUE,
    seed = 123
  )

  modelo_rf
}
```

```
#> Ranger result
#>
#> Call:
#> ranger(accidente_con_victimas ~ MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI, data =
#>
#> Type: Probability estimation
#> Number of trees: 200
#> Sample size: 21000
#> Number of independent variables: 5
#> Mtry: 3
#> Target node size: 20
#> Variable importance mode: impurity
#> Splitrule: gini
#> OOB prediction error (Brier s.): 0.1066668
```

Interpretación del resultado

El resumen del modelo muestra cuántos árboles se construyeron, cuántas variables se probaron en cada división y el error fuera de bolsa.

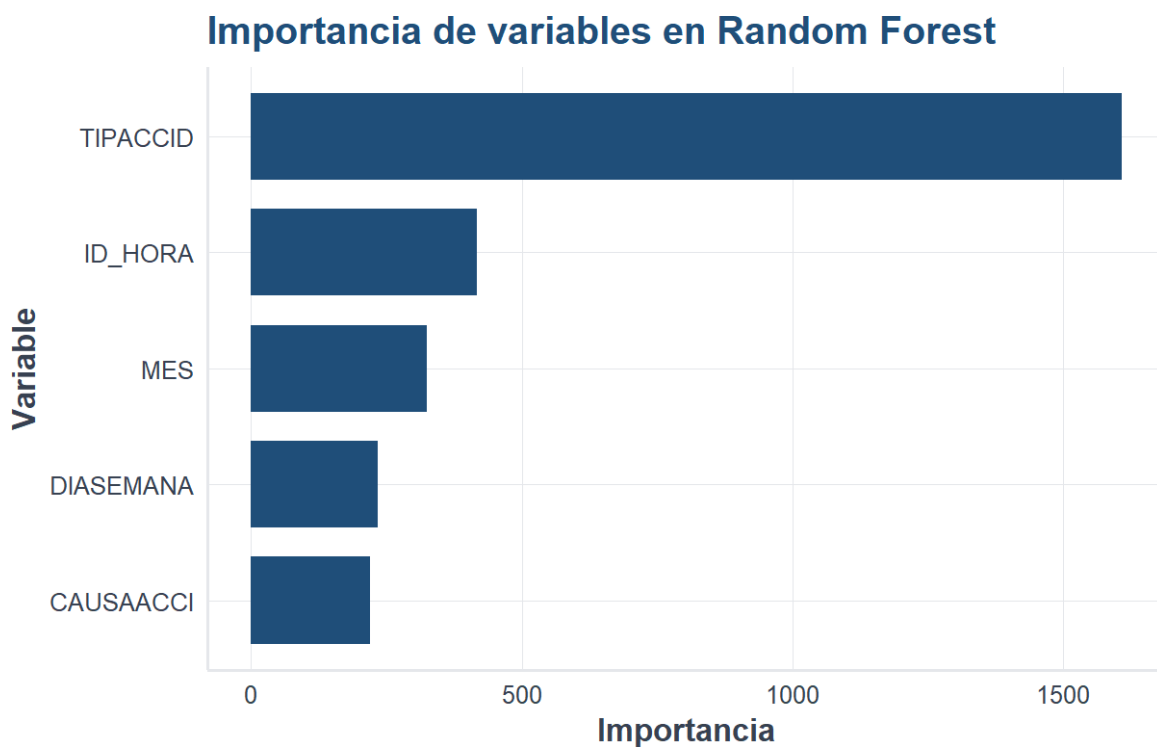
7.8 Importancia de variables

```
if (exists("modelo_rf")) {
  importancia_rf <- data.frame(
    variable = names(modelo_rf$variable.importance),
    importancia = as.numeric(modelo_rf$variable.importance)
  ) |>
  arrange(desc(importancia))
}
```

```
importancia_rf
}
```

```
#>   variable importancia
#> 1 TIPACCID    1608.8347
#> 2  ID_HORA     416.7870
#> 3     MES      324.9780
#> 4 DIASEMANA    234.4944
#> 5 CAUSAACCI    220.5923
```

```
if (exists("importancia_rf")) {
  ggplot(importancia_rf, aes(x = reorder(variable, importancia), y = importancia)) +
    geom_col(fill = col_azul, width = 0.75) +
    coord_flip() +
    labs(title = "Importancia de variables en Random Forest", x = "Variable", y = "Importancia")
  ↵ +
  tema_libro()
}
```



i Explicación del código

La importancia de variables muestra qué predictores aportan más a la reducción de impureza en los árboles del bosque.

7.9 Predicción y métricas

```
if (exists("modelo_rf")) {
  predicciones_rf <- predict(modelo_rf, data = prueba)
  probabilidades_rf <- predicciones_rf$predictions[, "Con víctimas"]
  clases_rf <- colnames(predicciones_rf$predictions)[max.col(predicciones_rf$predictions,
↪ ties.method = "first")]
  clases_rf <- factor(clases_rf, levels = c("Con víctimas", "Solo daños"))
  real_rf <- factor(prueba$accidente_con_victimas, levels = c("Con víctimas", "Solo daños"))

  matriz_confusion_rf <- table(Real = real_rf, Predicho = clases_rf)
  matriz_confusion_rf
}
```

```
#>               Predicho
#> Real           Con víctimas Solo daños
#> Con víctimas           475           991
#> Solo daños             265          7269
```

```
if (exists("matriz_confusion_rf")) {
  VP <- matriz_confusion_rf["Con víctimas", "Con víctimas"]
  FN <- matriz_confusion_rf["Con víctimas", "Solo daños"]
  FP <- matriz_confusion_rf["Solo daños", "Con víctimas"]
  VN <- matriz_confusion_rf["Solo daños", "Solo daños"]

  data.frame(exactitud=(VP+VN)/(VP+FN+FP+VN), sensibilidad=VP/(VP+FN), especificidad=VN/(VN+FP))
}
```

```
#> exactitud sensibilidad especificidad
#> 1 0.8604444      0.3240109      0.9648261
```

Interpretación del resultado

La sensibilidad indica qué tan bien detecta accidentes con víctimas. La especificidad indica qué tan bien reconoce accidentes de solo daños.

7.10 Distribución de probabilidades predichas

```
if (exists("probabilidades_rf")) {
  datos_prob_rf <- data.frame(probabilidad = probabilidades_rf, clase_real = real_rf)

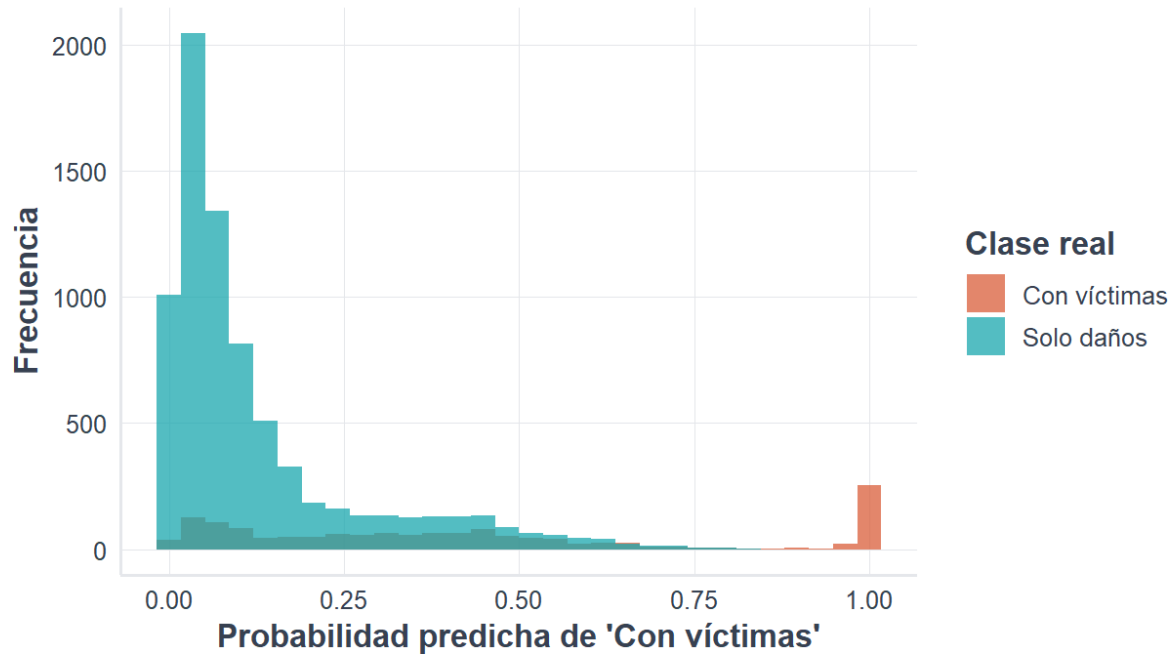
  ggplot(datos_prob_rf, aes(x = probabilidad, fill = clase_real)) +
    geom_histogram(bins = 30, alpha = 0.75, position = "identity") +
    escala_clases_fill(name = "Clase real") +
    labs(
      title = "Distribución de probabilidades predichas",
      subtitle = "Random Forest",
      x = "Probabilidad predicha de 'Con víctimas'",

```

```
y = "Frecuencia"
) +
tema_libro()
}
```

Distribución de probabilidades predichas

Random Forest



7.11 Conclusión

Random Forest mejora la estabilidad y la capacidad predictiva de un solo árbol al combinar muchos árboles. Es uno de los métodos más útiles para clasificación aplicada.

Capítulo 8

Evaluación, validación y comparación de modelos

8.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- explicar por qué la exactitud no siempre es suficiente;
- construir e interpretar una matriz de confusión;
- calcular exactitud, sensibilidad, especificidad, precisión y valor F1;
- distinguir entrenamiento, validación y prueba;
- aplicar validación cruzada estratificada;
- comparar modelos con criterios predictivos y prácticos;
- seleccionar un modelo sin depender de una sola métrica.

8.2 Introducción

Entrenar un modelo es apenas la mitad del trabajo. Después debemos responder una pregunta más delicada: **¿qué tan bien funcionará con accidentes que nunca vio durante el aprendizaje?**

Un modelo puede memorizar los datos de entrenamiento y parecer excelente, pero fallar con observaciones nuevas. Por eso la evaluación debe realizarse con datos separados y métricas apropiadas para el objetivo del problema.

En este capítulo se utiliza nuevamente la base preparada a partir de ATUS (Instituto Nacional de Estadística y Geografía 2025).

8.3 Cargar los datos

```
library(readr)
library(dplyr)
library(ggplot2)
source("util_graficas.R")

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (!file.exists(ruta_atus_ml)) {
  stop(
    paste(
      "No se encontró datos/atus_ml_preparado.csv.",
      "Renderice primero el capítulo de preparación de datos."
    )
  )
}
```

```

    )
  }

  atus_ml <- read_csv(
    ruta_atus_ml,
    show_col_types = FALSE
  )

```

8.4 Preparar una muestra reproducible

```

set.seed(123)

atus_eval <- atus_ml |>
  sample_n(min(30000, nrow(atus_ml))) |>
  mutate(
    accidente_con_victimas = factor(
      accidente_con_victimas,
      levels = c("Solo daños", "Con víctimas")
    ),
    MES = factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = factor(DIASEMANA),
    TIPACCID = factor(TIPACCID),
    CAUSAACCI = factor(CAUSAACCI)
  ) |>
  na.omit()

prop.table(table(atus_eval$accidente_con_victimas))

```

```

#>
#> Solo daños Con víctimas
#> 0.8277333 0.1722667

```

Interpretación

La tabla de proporciones permite identificar si una clase es mucho más frecuente que la otra. Cuando existe desbalance, una exactitud alta puede resultar engañosa.

8.5 Entrenamiento, validación y prueba

Los datos pueden dividirse en tres partes:

- **Entrenamiento:** se utiliza para estimar los parámetros del modelo.
- **Validación:** ayuda a elegir hiperparámetros y comparar alternativas.
- **Prueba:** se reserva para estimar el desempeño final.

En ejercicios introductorios es frecuente utilizar entrenamiento y prueba, mientras que la validación se realiza mediante validación cruzada dentro del conjunto de entrenamiento.

8.6 División estratificada

Una división estratificada conserva aproximadamente la proporción de cada clase.

```
set.seed(123)

indices_entrenamiento <- unlist(
  lapply(
    split(seq_len(nrow(atus_eval)), atus_eval$accidente_con_victimas),
    function(indices) {
      sample(indices, size = floor(0.70 * length(indices)))
    }
  )
)

entrenamiento <- atus_eval[indices_entrenamiento, ]
prueba <- atus_eval[-indices_entrenamiento, ]

prop.table(table(entrenamiento$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#> 0.8277537 0.1722463
```

```
prop.table(table(prueba$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#> 0.8276858 0.1723142
```

Explicación del código

Los índices se separan por clase y después se toma 70 % de cada grupo. Así se reduce el riesgo de que una clase quede sobrerrepresentada en entrenamiento o prueba.

8.7 Matriz de confusión

Para una clase positiva denominada **Con víctimas**, la matriz contiene:

- **Verdadero positivo (VP)**: el accidente tenía víctimas y el modelo lo detectó.
- **Falso negativo (FN)**: tenía víctimas, pero el modelo lo clasificó como solo daños.
- **Falso positivo (FP)**: era de solo daños, pero el modelo predijo víctimas.
- **Verdadero negativo (VN)**: era de solo daños y se clasificó correctamente.

Clase real	Predicción positiva	Predicción negativa
Positiva	VP	FN
Negativa	FP	VN

Un falso negativo puede ser especialmente importante cuando el propósito consiste en identificar accidentes con víctimas.

8.8 Métricas principales

8.8.1 Exactitud

$$\text{Exactitud} = \frac{VP + VN}{VP + FN + FP + VN}$$

Mide la proporción total de predicciones correctas.

8.8.2 Sensibilidad

$$\text{Sensibilidad} = \frac{VP}{VP + FN}$$

Mide qué proporción de los accidentes con víctimas fue detectada.

8.8.3 Especificidad

$$\text{Especificidad} = \frac{VN}{VN + FP}$$

Mide qué proporción de los accidentes de solo daños fue reconocida correctamente.

8.8.4 Precisión

$$\text{Precisión} = \frac{VP}{VP + FP}$$

Indica qué proporción de las predicciones positivas realmente tenía víctimas.

8.8.5 Valor F1

$$F_1 = 2 \frac{\text{Precisión} \times \text{Sensibilidad}}{\text{Precisión} + \text{Sensibilidad}}$$

El valor F1 equilibra precisión y sensibilidad.

8.9 Función para calcular métricas

```

calcular_metricas <- function(real, predicho, positiva = "Con víctimas") {
  real <- factor(real)
  predicho <- factor(predicho, levels = levels(real))

  negativas <- setdiff(levels(real), positiva)

  if (length(negativas) != 1) {
    stop("La función requiere exactamente dos clases.")
  }

  negativa <- negativas[1]
  matriz <- table(Real = real, Predicho = predicho)

  VP <- matriz[positiva, positiva]
  FN <- matriz[positiva, negativa]
  FP <- matriz[negativa, positiva]
  VN <- matriz[negativa, negativa]

  division_segura <- function(numerador, denominador) {
    if (
      is.na(numerador) ||
      is.na(denominador) ||
      denominador == 0
    ) {
      return(NA_real_)
    }

    as.numeric(numerador / denominador)
  }

  exactitud <- division_segura(VP + VN, VP + FN + FP + VN)
  sensibilidad <- division_segura(VP, VP + FN)
  especificidad <- division_segura(VN, VN + FP)
  precision <- division_segura(VP, VP + FP)
  f1 <- division_segura(
    2 * precision * sensibilidad,
    precision + sensibilidad
  )

  list(
    matriz = matriz,
    metricas = data.frame(
      exactitud = exactitud,
      sensibilidad = sensibilidad,
      especificidad = especificidad,
      precision = precision,
      f1 = f1
    )
  )
}

```

8.10 Modelo de referencia: regla mayoritaria

Antes de celebrar un modelo complejo, conviene compararlo con una regla muy simple: predecir siempre la clase más frecuente.

```
clase_mayoritaria <- names(
  which.max(table(entrenamiento$accidente_con_victimas))
)

prediccion_referencia <- factor(
  rep(clase_mayoritaria, nrow(prueba)),
  levels = levels(entrenamiento$accidente_con_victimas)
)

resultado_referencia <- calcular_metricas(
  real = prueba$accidente_con_victimas,
  predicho = prediccion_referencia
)

resultado_referencia$matriz
```

```
#>               Predicho
#> Real              Solo daños Con víctimas
#> Solo daños           7450           0
#> Con víctimas        1551           0
```

```
resultado_referencia$metricas
```

```
#> exactitud sensibilidad especificidad precision f1
#> 1 0.8276858           0              1      NA NA
```

¿Por qué puede aparecer NA?

El modelo de referencia predice únicamente la clase mayoritaria. Si nunca predice la clase positiva **Con víctimas**, la precisión no puede calcularse porque su denominador es cero. En ese caso R muestra **NA**, que significa **métrica no definida**, pero el libro continúa procesándose normalmente.

Una exactitud alta puede engañar

Si 90 % de los accidentes perteneciera a una sola clase, predecir siempre esa clase produciría 90 % de exactitud sin aprender ninguna relación útil. Por eso deben revisarse sensibilidad, especificidad, precisión y F1.

8.11 Evaluar una regresión logística

```

modelo_logistico <- glm(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = entrenamiento,
  family = binomial
)

probabilidad_logistica <- predict(
  modelo_logistico,
  newdata = prueba,
  type = "response"
)

# glm() modela la probabilidad del segundo nivel del factor.
clase_positiva_modelada <- levels(
  entrenamiento$accidente_con_victimas
)[2]

prediccion_logistica <- ifelse(
  probabilidad_logistica >= 0.50,
  clase_positiva_modelada,
  levels(entrenamiento$accidente_con_victimas)[1]
)

prediccion_logistica <- factor(
  prediccion_logistica,
  levels = levels(entrenamiento$accidente_con_victimas)
)

resultado_logistico <- calcular_metricas(
  real = prueba$accidente_con_victimas,
  predicho = prediccion_logistica
)

resultado_logistico$matriz

```

```

#>               Predicho
#> Real              Solo daños Con víctimas
#> Solo daños          7403          47
#> Con víctimas        1198         353

```

```
resultado_logistico$metricas
```

```

#> exactitud sensibilidad especificidad precision      f1
#> 1  0.861682    0.2275951    0.9936913    0.8825 0.3618657

```

! Revise qué clase modela R

En una regresión logística binaria, `glm()` calcula la probabilidad del segundo nivel del factor respuesta. El orden de los niveles debe verificarse antes de transformar probabilidades en clases.

8.12 Comparación inicial

```
comparacion_inicial <- bind_rows(
  cbind(
    modelo = "Regla mayoritaria",
    resultado_referencia$metricas
  ),
  cbind(
    modelo = "Regresión logística",
    resultado_logistico$metricas
  )
)
comparacion_inicial
```

```
#>           modelo exactitud sensibilidad especificidad precision      f1
#> 1  Regla mayoritaria 0.8276858    0.0000000    1.0000000      NA      NA
#> 2 Regresión logística 0.8616820    0.2275951    0.9936913    0.8825 0.3618657
```

```
comparacion_larga <- comparacion_inicial |>
  tidyr::pivot_longer(
    cols = -modelo,
    names_to = "metrica",
    values_to = "valor"
  )

ggplot(
  comparacion_larga,
  aes(x = metrica, y = valor, fill = modelo)
) +
  geom_col(position = "dodge") +
  scale_y_continuous(
    limits = c(0, 1),
    labels = scales::label_percent(accuracy = 1)
  ) +
  labs(
    title = "Un modelo debe superar una referencia sencilla",
    x = "Métrica",
    y = "Valor",
    fill = "Modelo"
  ) +
  tema_libro()
```

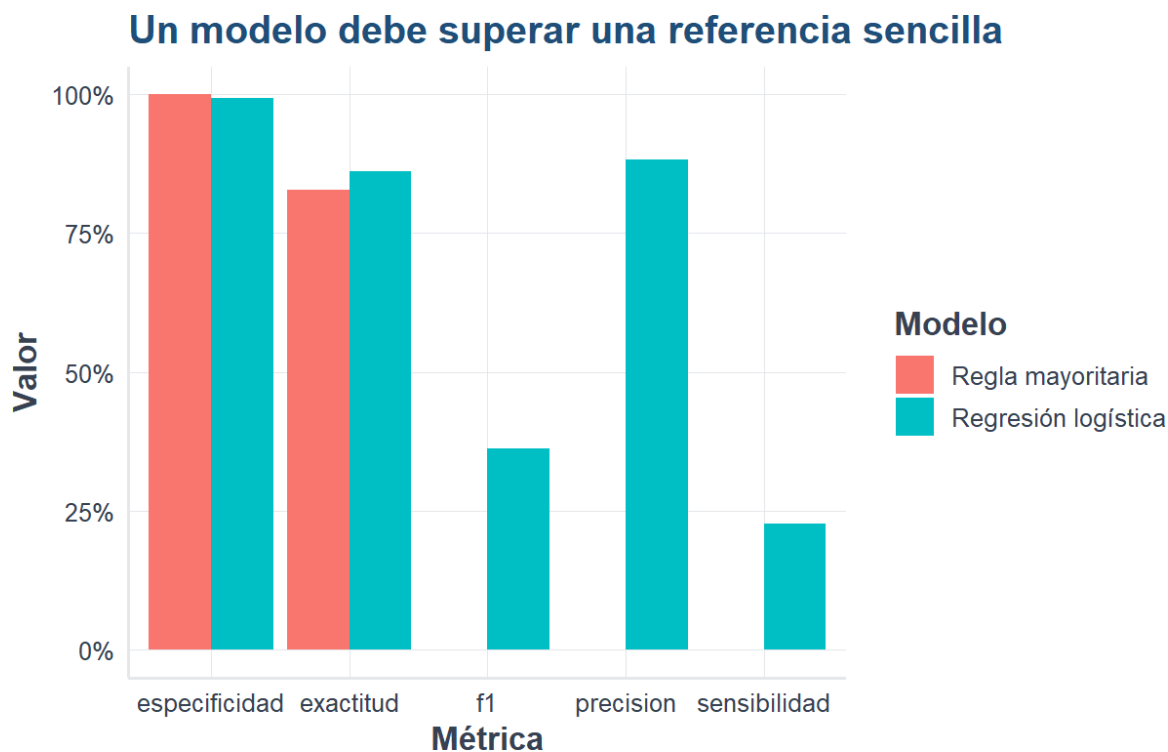


Figura 8.1: Comparación de métricas entre una regla de referencia y la regresión logística.

Fuente: Elaboración propia con datos del INEGI, ATUS 2024.

8.13 Umbral de clasificación

El umbral de 0.50 no es una ley universal. Reducirlo puede aumentar la sensibilidad, aunque también puede generar más falsos positivos.

```

umbrales <- seq(0.10, 0.90, by = 0.05)

metricas_umbral <- lapply(umbrales, function(umbral) {
  prediccion <- ifelse(
    probabilidad_logistica >= umbral,
    clase_positiva_modelada,
    levels(entrenamiento$accidente_con_victimas)[1]
  )

  prediccion <- factor(
    prediccion,
    levels = levels(entrenamiento$accidente_con_victimas)
  )

  metricas <- calcular_metricas(
    prueba$accidente_con_victimas,
    prediccion
  )$metricas

  cbind(umbral = umbral, metricas)
}) |>
    
```

```
bind_rows()

metricas_umbral
```

```
#>      umbral exactitud sensibilidad especificidad precision      f1
#> 1      0.10 0.7351405      0.7878788      0.7241611 0.3729020 0.5062138
#> 2      0.15 0.7996889      0.7150226      0.8173154 0.4489879 0.5516041
#> 3      0.20 0.8055772      0.7047066      0.8265772 0.4582809 0.5553862
#> 4      0.25 0.8182424      0.6834300      0.8463087 0.4807256 0.5644302
#> 5      0.30 0.8196867      0.6756931      0.8496644 0.4833948 0.5635924
#> 6      0.35 0.8237974      0.6434558      0.8613423 0.4913836 0.5572306
#> 7      0.40 0.8447950      0.4455190      0.9279195 0.5627036 0.4973012
#> 8      0.45 0.8586824      0.2778852      0.9795973 0.7392796 0.4039363
#> 9      0.50 0.8616820      0.2275951      0.9936913 0.8825000 0.3618657
#> 10     0.55 0.8602378      0.2127660      0.9950336 0.8991826 0.3441084
#> 11     0.60 0.8593490      0.1889104      0.9989262 0.9734219 0.3164147
#> 12     0.65 0.8596823      0.1856867      1.0000000 1.0000000 0.3132137
#> 13     0.70 0.8596823      0.1856867      1.0000000 1.0000000 0.3132137
#> 14     0.75 0.8596823      0.1856867      1.0000000 1.0000000 0.3132137
#> 15     0.80 0.8596823      0.1856867      1.0000000 1.0000000 0.3132137
#> 16     0.85 0.8596823      0.1856867      1.0000000 1.0000000 0.3132137
#> 17     0.90 0.8596823      0.1856867      1.0000000 1.0000000 0.3132137
```

i Decisión práctica

El mejor umbral depende del costo de cada error. Cuando omitir un accidente con víctimas es más grave que generar una alerta adicional, puede priorizarse la sensibilidad.

8.14 Validación cruzada

La validación cruzada divide el conjunto de entrenamiento en varios pliegues. Cada pliegue funciona una vez como validación y los restantes como entrenamiento.

En validación cruzada de K pliegues:

1. se divide la muestra en K partes;
2. se entrena con $K - 1$ partes;
3. se evalúa en la parte restante;
4. se repite hasta utilizar todos los pliegues;
5. se promedian las métricas.

8.14.1 Crear pliegues estratificados

```
crear_pliegues <- function(respuesta, k = 5, semilla = 123) {
  set.seed(semilla)
```

```

pliegues <- integer(length(respuesta))

for (clase in levels(factor(respuesta))) {
  indices <- which(respuesta == clase)
  etiquetas <- rep(seq_len(k), length.out = length(indices))
  pliegues[indices] <- sample(etiquetas)
}

pliegues
}

pliegue <- crear_pliegues(
  entrenamiento$accidente_con_victimas,
  k = 5
)

table(pliegue, entrenamiento$accidente_con_victimas)

```

```

#>
#> pliegue Solo daños Con víctimas
#>      1      3477      724
#>      2      3477      724
#>      3      3476      723
#>      4      3476      723
#>      5      3476      723

```

8.14.2 Validación cruzada de la regresión logística

```

resultados_cv <- lapply(1:5, function(k) {
  datos_entrenamiento <- entrenamiento[pliegue != k, ]
  datos_validacion <- entrenamiento[pliegue == k, ]

  modelo <- glm(
    accidente_con_victimas ~
      MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
    data = datos_entrenamiento,
    family = binomial
  )

  probabilidad <- predict(
    modelo,
    newdata = datos_validacion,
    type = "response"
  )

  clase_modelada <- levels(
    datos_entrenamiento$accidente_con_victimas
  )[2]

  prediccion <- ifelse(
    probabilidad >= 0.50,
    clase_modelada,
    levels(datos_entrenamiento$accidente_con_victimas)[1]
  )
})

```



```

prediccion <- factor(
  prediccion,
  levels = levels(datos_entrenamiento$accidente_con_victimas)
)

metricas <- calcular_metricas(
  datos_validacion$accidente_con_victimas,
  prediccion
)$metricas

cbind(pliege = k, metricas)
}) |>
  bind_rows()

resultados_cv

```

```

#>   pliege exactitud sensibilidad especificidad precision      f1
#> 1     1 0.8631278    0.2389503    0.9930975 0.8781726 0.3756786
#> 2     2 0.8590812    0.2140884    0.9933851 0.8707865 0.3436807
#> 3     3 0.8630626    0.2268326    0.9953970 0.9111111 0.3632337
#> 4     4 0.8637771    0.2406639    0.9933832 0.8832487 0.3782609
#> 5     5 0.8525839    0.1798064    0.9925201 0.8333333 0.2957907

```

```

resumen_cv <- resultados_cv |>
  summarise(
    across(
      where(is.numeric) & !matches("pliege"),
      list(media = mean, desviacion = sd),
      na.rm = TRUE
    )
  )

resumen_cv

```

```

#>   exactitud_media exactitud_desviacion sensibilidad_media
#> 1      0.8603265      0.004710052      0.2200683
#>   sensibilidad_desviacion especificidad_media especificidad_desviacion
#> 1      0.02491609      0.9935566      0.001087613
#>   precision_media precision_desviacion f1_media f1_desviacion
#> 1      0.8753305      0.02799749 0.3513289      0.03392253

```

Interpretación

La media resume el desempeño esperado y la desviación estándar muestra su estabilidad. Dos modelos con medias parecidas pueden diferir mucho si uno presenta resultados más variables entre pliegues.

8.15 Cómo comparar varios modelos

Para comparar regresión logística, k-NN, árbol de decisión y Random Forest conviene utilizar:

1. la misma muestra;
2. la misma variable respuesta;
3. la misma partición de entrenamiento y prueba;
4. el mismo criterio para definir la clase positiva;
5. las mismas métricas;
6. una validación cruzada equivalente;
7. tiempos de entrenamiento y predicción;
8. facilidad de interpretación.

Una tabla de decisión puede tener esta estructura:

Modelo	Exactitud	Sensibilidad	Especificidad	F1	Interpretación	Costo computacional
Regresión logística					Alta	Bajo
k-NN					Media-baja	Medio
Árbol de decisión					Alta	Bajo
Random Forest					Media	Alto

8.16 Selección del modelo final

No existe un modelo universalmente mejor. La selección depende de la finalidad:

- Si se requiere explicar el efecto de las variables, puede preferirse regresión logística.
- Si se necesita una regla visual y fácil de comunicar, un árbol puede ser apropiado.
- Si se prioriza capacidad predictiva y estabilidad, Random Forest puede resultar competitivo.
- Si el costo de los falsos negativos es alto, debe priorizarse sensibilidad.
- Si las alertas falsas son costosas, precisión y especificidad cobran mayor importancia.

! Principio fundamental

El mejor modelo no es necesariamente el que tiene la mayor exactitud, sino el que responde mejor al objetivo, controla los errores relevantes y mantiene un desempeño estable con datos nuevos.

8.17 Actividad guiada

1. Ejecute la regresión logística con umbrales de 0.30, 0.50 y 0.70.
2. Registre la matriz de confusión de cada caso.
3. Compare sensibilidad, especificidad, precisión y F1.
4. Explique qué umbral elegiría para detectar accidentes con víctimas.
5. Justifique su decisión considerando el costo de falsos negativos y falsos positivos.

8.18 Actividades para el lector

1. Construya una función que calcule la exactitud balanceada:

$$\text{Exactitud balanceada} = \frac{\text{Sensibilidad} + \text{Especificidad}}{2}$$

2. Compare la variabilidad de las métricas en validación cruzada.
3. Repita el procedimiento con un árbol de decisión.
4. Elabore una tabla comparativa con los cuatro modelos estudiados.
5. Escriba una recomendación técnica de no más de 200 palabras.

8.19 Resumen del capítulo

En este capítulo aprendimos que evaluar un modelo requiere datos no utilizados durante el entrenamiento, una clase positiva bien definida y varias métricas. También construimos una regla de referencia, estudiamos el efecto del umbral y aplicamos validación cruzada estratificada. Estas herramientas permiten comparar modelos de forma más justa y elegirlos según el objetivo real del análisis.

Capítulo 9

Máquinas de Vectores de Soporte (SVM)

9.1 Objetivos del capítulo

Al finalizar este capítulo, el lector será capaz de:

- explicar la idea del hiperplano de separación y del margen máximo;
- identificar los vectores de soporte;
- distinguir entre una SVM lineal y una SVM con kernel;
- comprender el papel de los parámetros `cost`, `gamma` y del tipo de kernel;
- entrenar modelos SVM en R;
- evaluar una SVM mediante una matriz de confusión y métricas de clasificación;
- utilizar ponderación de clases cuando la variable respuesta está desbalanceada;
- comparar una SVM lineal con una SVM de kernel radial.

9.2 Introducción

Las **máquinas de vectores de soporte**, conocidas como **SVM** por *Support Vector Machines*, son métodos supervisados utilizados principalmente para clasificación. Su propósito es construir una frontera que separe las clases con el margen más amplio posible.

La idea es sencilla de visualizar: si dos grupos pueden separarse mediante una línea, existen muchas líneas posibles, pero la SVM busca aquella que deja la mayor distancia entre la frontera y las observaciones más cercanas de cada clase.

Esas observaciones cercanas reciben el nombre de **vectores de soporte**. Aunque el conjunto de datos contenga miles de registros, los vectores de soporte son los que determinan directamente la posición de la frontera.

En este capítulo se emplean primero datos simulados para entender la geometría del método y después la base preparada de ATUS (Instituto Nacional de Estadística y Geografía 2025).

9.3 Intuición geométrica

Supongamos que cada observación tiene dos variables, x_1 y x_2 . Una frontera lineal puede escribirse como:

$$w_1x_1 + w_2x_2 + b = 0$$

donde:

- w_1 y w_2 determinan la orientación de la frontera;

- b desplaza la frontera;
- el signo de la expresión determina de qué lado queda cada observación.

La SVM busca una frontera con **margen máximo**. En una separación ideal se desea que:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

para todas las observaciones, donde y_i representa la clase codificada como -1 o 1 .

El ancho del margen es proporcional a:

$$\frac{2}{\|\mathbf{w}\|}$$

Por ello, maximizar el margen equivale a minimizar la magnitud de $\mathbf{w}$.

Idea esencial

Una SVM no busca solamente clasificar bien los datos de entrenamiento. Busca una frontera que conserve la mayor separación posible entre las clases, con la intención de generalizar mejor ante observaciones nuevas.

9.4 Crear un ejemplo didáctico

```
set.seed(123)

n <- 120

datos_svm <- data.frame(
  x1 = c(rnorm(n / 2, mean = -1.5, sd = 0.8),
        rnorm(n / 2, mean = 1.5, sd = 0.8)),
  x2 = c(rnorm(n / 2, mean = -1.0, sd = 0.9),
        rnorm(n / 2, mean = 1.0, sd = 0.9)),
  clase = factor(rep(c("Clase A", "Clase B"), each = n / 2))
)

head(datos_svm)
```

```
#>           x1           x2  clase
#> 1 -1.9483805 -0.8941181 Clase A
#> 2 -1.6841420 -1.8527272 Clase A
#> 3 -0.2530333 -1.4415017 Clase A
#> 4 -1.4435933 -1.2304830 Clase A
#> 5 -1.3965698  0.6594758 Clase A
#> 6 -0.1279480 -1.5867549 Clase A
```

i Explicación del código

Se generan dos grupos artificiales. El primero se concentra alrededor de valores negativos y el segundo alrededor de valores positivos. El ejemplo permite observar con claridad la frontera de clasificación.

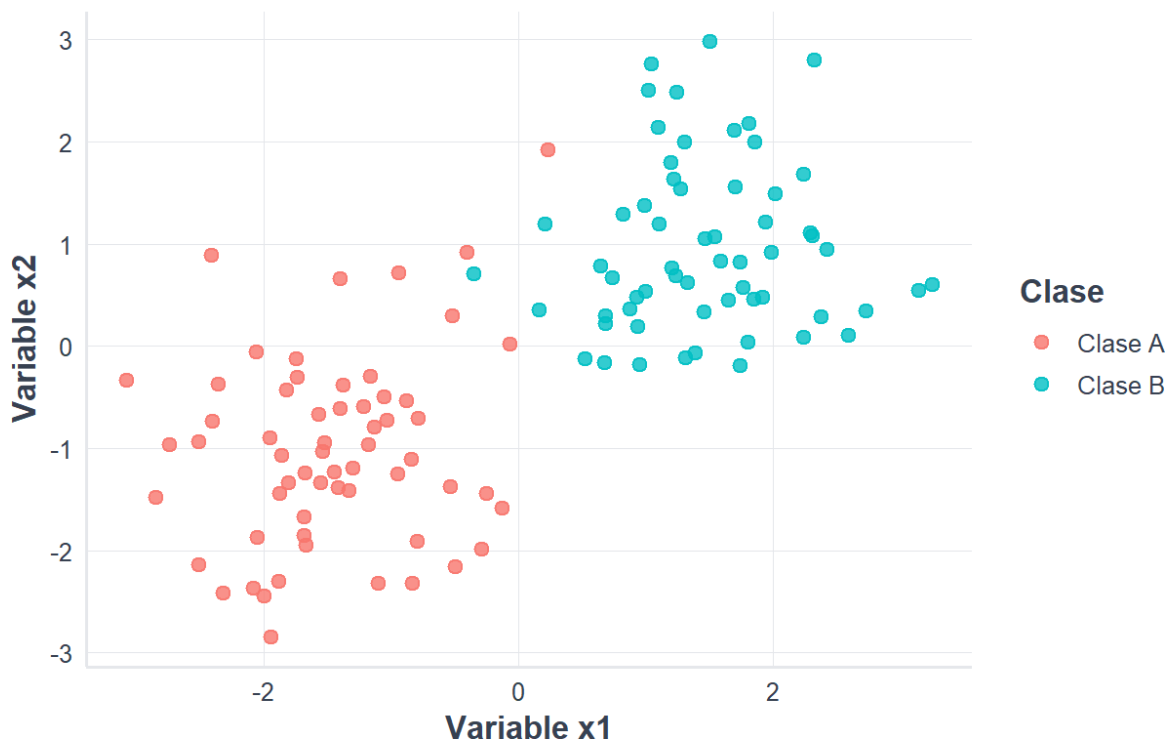
9.5 Visualizar las clases

```
library(ggplot2)
source("util_graficas.R")

ggplot(datos_svm, aes(x = x1, y = x2, color = clase)) +
  geom_point(size = 2.5, alpha = 0.8) +
  labs(
    title = "Datos simulados para clasificación",
    subtitle = "Cada punto representa una observación",
    x = "Variable x1",
    y = "Variable x2",
    color = "Clase"
  ) +
  tema_libro()
```

Datos simulados para clasificación

Cada punto representa una observación



9.6 Instalar y cargar el paquete e1071

```
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071", repos = "https://cloud.r-project.org")
}

library(e1071)
```

El paquete e1071 proporciona la función `svm()`, que permite entrenar modelos lineales y no lineales.

9.7 Entrenar una SVM lineal

```
modelo_svm_lineal <- svm(
  clase ~ x1 + x2,
  data = datos_svm,
  kernel = "linear",
  cost = 1,
  scale = TRUE
)

modelo_svm_lineal
```

```
#>
#> Call:
#> svm(formula = clase ~ x1 + x2, data = datos_svm, kernel = "linear",
#>      cost = 1, scale = TRUE)
#>
#>
#> Parameters:
#>   SVM-Type:  C-classification
#>   SVM-Kernel: linear
#>      cost:   1
#>
#> Number of Support Vectors: 14
```

Los argumentos principales son:

- `kernel = "linear"`: utiliza una frontera lineal;
- `cost = 1`: establece la penalización por errores;
- `scale = TRUE`: estandariza las variables numéricas antes del ajuste.

9.8 Identificar los vectores de soporte

```
indices_soporte <- modelo_svm_lineal$index
vectores_soporte <- datos_svm[indices_soporte, ]

nrow(vectores_soporte)
```

```
#> [1] 14
```

```
head(vectores_soporte)
```

```
#>           x1           x2  clase
#> 3  -0.25303335 -1.44150170 Clase A
#> 6  -0.12794801 -1.58675491 Clase A
#> 11 -0.52073456  0.30009577 Clase A
#> 16 -0.07046949  0.01820349 Clase A
#> 19 -0.93891528  0.71819321 Clase A
#> 44  0.23516477  1.91693594 Clase A
```

Interpretación

Los vectores de soporte son las observaciones más influyentes para establecer la frontera. Los puntos alejados del margen suelen tener poca o ninguna influencia directa sobre su posición.

9.9 Dibujar la frontera de decisión

```
rejilla <- expand.grid(
  x1 = seq(min(datos_svm$x1) - 0.5,
            max(datos_svm$x1) + 0.5,
            length.out = 180),
  x2 = seq(min(datos_svm$x2) - 0.5,
            max(datos_svm$x2) + 0.5,
            length.out = 180)
)

rejilla$prediccion <- predict(modelo_svm_lineal, newdata = rejilla)

ggplot() +
  geom_tile(
    data = rejilla,
    aes(x = x1, y = x2, fill = prediccion),
    alpha = 0.18
  ) +
  geom_point(
    data = datos_svm,
    aes(x = x1, y = x2, color = clase),
    size = 2.2
```



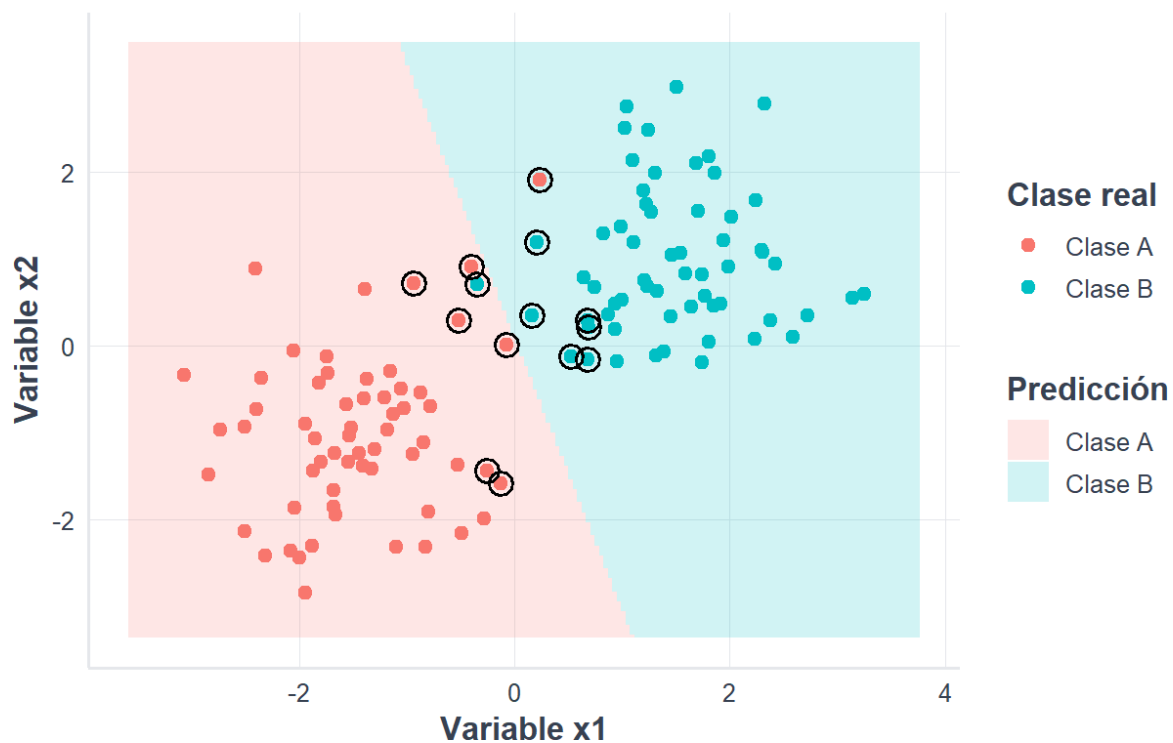
```

) +
geom_point(
  data = vectores_soporte,
  aes(x = x1, y = x2),
  shape = 21,
  size = 4,
  stroke = 1.1,
  fill = NA
) +
labs(
  title = "Frontera de decisión de una SVM lineal",
  subtitle = "Los círculos grandes identifican los vectores de soporte",
  x = "Variable x1",
  y = "Variable x2",
  fill = "Predicción",
  color = "Clase real"
) +
tema_libro()

```

Frontera de decisión de una SVM lineal

Los círculos grandes identifican los vectores de soporte



9.10 El parámetro de costo

El parámetro `cost` controla la penalización asignada a las observaciones clasificadas incorrectamente.

- Un costo pequeño permite más errores y favorece un margen amplio.
- Un costo grande penaliza fuertemente los errores y puede producir una frontera más ajustada a los datos de entrenamiento.

No existe un valor universalmente mejor. Debe seleccionarse con validación cruzada.

```
modelos_cost <- lapply(
  c(0.1, 1, 10),
  function(valor_cost) {
    svm(
      clase ~ x1 + x2,
      data = datos_svm,
      kernel = "linear",
      cost = valor_cost,
      scale = TRUE
    )
  }
)

resumen_cost <- data.frame(
  cost = c(0.1, 1, 10),
  vectores_soporte = vapply(
    modelos_cost,
    function(modelo) modelo$tot.nSV,
    numeric(1)
  )
)

resumen_cost
```

```
#>   cost vectores_soporte
#> 1  0.1                30
#> 2  1.0                14
#> 3 10.0                 8
```

9.11 Cuando una línea no es suficiente

Algunos conjuntos de datos no pueden separarse adecuadamente mediante una línea o un hiperplano. En esos casos, una SVM puede utilizar una función denominada **kernel**.

El kernel calcula similitudes entre observaciones y permite representar fronteras no lineales sin construir explícitamente todas las nuevas variables.

Los kernels más utilizados son:

Kernel	Uso general
Lineal	Cuando la separación es aproximadamente lineal o existen muchas variables
Polinomial	Cuando se esperan relaciones de tipo polinómico
Radial	Cuando la frontera puede tener formas curvas y complejas
Sigmoide	Menos frecuente; guarda relación con funciones de activación

9.12 Kernel radial

El kernel radial utiliza una función semejante a:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

El parámetro γ controla el alcance de la influencia de cada observación:

- valores pequeños producen fronteras más suaves;
- valores grandes permiten fronteras más locales y complejas.

Riesgo de sobreajuste

Una combinación de `cost` y `gamma` demasiado grande puede ajustar muy bien el entrenamiento, pero funcionar peor con datos nuevos. La validación cruzada ayuda a evitar esa elección.

Capítulo 10

Aplicación con los datos ATUS

10.1 Cargar la base preparada

```
library(readr)
library(dplyr)

ruta_atus_ml <- "datos/atus_ml_preparado.csv"

if (!file.exists(ruta_atus_ml)) {
  stop(
    paste(
      "No se encontró datos/atus_ml_preparado.csv.",
      "Renderice primero el capítulo de preparación de datos."
    )
  )
}

atus_ml <- read_csv(
  ruta_atus_ml,
  show_col_types = FALSE
)
```

10.2 Preparar una muestra reproducible

Las SVM pueden requerir bastante memoria y tiempo cuando el conjunto es muy grande. Para fines didácticos se utiliza una muestra de hasta 6 000 accidentes.

```
set.seed(123)

atus_svm <- atus_ml |>
  sample_n(min(6000, nrow(atus_ml))) |>
  transmute(
    accidente_con_victimas = factor(
      accidente_con_victimas,
      levels = c("Solo daños", "Con víctimas")
    ),
    MES = factor(MES),
    ID_HORA = as.numeric(ID_HORA),
    DIASEMANA = factor(DIASEMANA),
    TIPACCID = factor(TIPACCID),
    CAUSAACCI = factor(CAUSAACCI)
  ) |>
  na.omit()
```

```
prop.table(table(atus_svm$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#>      0.82      0.18
```

i Nota sobre la muestra

La muestra reduce el tiempo de ejecución y hace posible repetir el ejercicio en computadoras personales y en Google Colab. Para un estudio definitivo se recomienda evaluar tamaños mayores y documentar los recursos computacionales utilizados.

10.3 Dividir los datos de forma estratificada

```
set.seed(123)

indices_entrenamiento <- unlist(
  lapply(
    split(seq_len(nrow(atus_svm)), atus_svm$accidente_con_victimas),
    function(indices) {
      sample(indices, size = floor(0.70 * length(indices)))
    }
  )
)

entrenamiento_svm <- atus_svm[indices_entrenamiento, ]
prueba_svm <- atus_svm[-indices_entrenamiento, ]

prop.table(table(entrenamiento_svm$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#>      0.82      0.18
```

```
prop.table(table(prueba_svm$accidente_con_victimas))
```

```
#>
#> Solo daños Con víctimas
#>      0.82      0.18
```

10.4 Calcular pesos para las clases

Si una clase aparece con menor frecuencia, la SVM puede recibir pesos inversamente proporcionales a sus frecuencias.

```
frecuencias <- table(entrenamiento_svm$accidente_con_victimas)

pesos_clase <- sum(frecuencias) /
  (length(frecuencias) * frecuencias)

pesos_clase <- as.numeric(pesos_clase)
names(pesos_clase) <- names(frecuencias)

pesos_clase
```

```
#> Solo daños Con víctimas
#> 0.6097561 2.7777778
```

La clase menos frecuente recibe un peso mayor, de modo que sus errores tengan más influencia durante el entrenamiento.

10.5 Ajustar una SVM lineal

```
set.seed(123)

modelo_svm_atus_lineal <- svm(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = entrenamiento_svm,
  kernel = "linear",
  cost = 1,
  class.weights = pesos_clase,
  scale = TRUE
)

modelo_svm_atus_lineal
```

```
#>
#> Call:
#> svm(formula = accidente_con_victimas ~ MES + ID_HORA + DIASEMANA +
#>      TIPACCID + CAUSAACCI, data = entrenamiento_svm, kernel = "linear",
#>      cost = 1, class.weights = pesos_clase, scale = TRUE)
#>
#>
#> Parameters:
#>   SVM-Type:  C-classification
#>   SVM-Kernel: linear
#>      cost:   1
#>
#> Number of Support Vectors: 2215
```

10.6 Realizar predicciones

```
prediccion_svm_lineal <- predict(
  modelo_svm_atu_lineal,
  newdata = prueba_svm
)

matriz_svm_lineal <- table(
  Real = prueba_svm$accidente_con_victimas,
  Predicho = prediccion_svm_lineal
)

matriz_svm_lineal
```

```
#>               Predicho
#> Real              Solo daños Con víctimas
#> Solo daños          1172          304
#> Con víctimas         89          235
```

10.7 Función segura para calcular métricas

```
metricas_clasificacion <- function(
  real,
  predicho,
  positiva = "Con víctimas"
) {
  real <- factor(real)
  predicho <- factor(predicho, levels = levels(real))

  negativa <- setdiff(levels(real), positiva)

  if (length(negativa) != 1) {
    stop("La función requiere exactamente dos clases.")
  }

  negativa <- negativa[1]
  matriz <- table(Real = real, Predicho = predicho)

  VP <- matriz[positiva, positiva]
  FN <- matriz[positiva, negativa]
  FP <- matriz[negativa, positiva]
  VN <- matriz[negativa, negativa]

  dividir <- function(a, b) {
    if (is.na(a) || is.na(b) || b == 0) {
      return(NA_real_)
    }
    as.numeric(a / b)
  }

  exactitud <- dividir(VP + VN, VP + FN + FP + VN)
  sensibilidad <- dividir(VP, VP + FN)
  especificidad <- dividir(VN, VN + FP)
```

```
precision <- dividir(VP, VP + FP)
f1 <- dividir(
  2 * precision * sensibilidad,
  precision + sensibilidad
)

data.frame(
  exactitud = exactitud,
  sensibilidad = sensibilidad,
  especificidad = especificidad,
  precision = precision,
  f1 = f1
)
```

10.8 Evaluar la SVM lineal

```
metricas_svm_lineal <- metricas_clasificacion(
  real = prueba_svm$accidente_con_victimas,
  predicho = prediccion_svm_lineal
)

metricas_svm_lineal
```

```
#>   exactitud sensibilidad especificidad precision      f1
#> 1 0.7816667    0.7253086    0.7940379 0.4359926 0.5446118
```

Interpretación

La sensibilidad indica la proporción de accidentes con víctimas detectada por el modelo. La especificidad indica la proporción de accidentes de solo daños reconocida correctamente. Cuando las clases están desbalanceadas, estas métricas suelen ser más informativas que la exactitud aislada.

10.9 Ajustar una SVM con kernel radial

```
set.seed(123)

modelo_svm_atus_radial <- svm(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = entrenamiento_svm,
  kernel = "radial",
  cost = 1,
  gamma = 0.03,
  class.weights = pesos_clase,
  scale = TRUE
)
```



```
modelo_svm_atus_radial
```

```
#>
#> Call:
#> svm(formula = accidente_con_victimas ~ MES + ID_HORA + DIASEMANA +
#>      TIPACCID + CAUSAACCI, data = entrenamiento_svm, kernel = "radial",
#>      cost = 1, gamma = 0.03, class.weights = pesos_clase, scale = TRUE)
#>
#>
#> Parameters:
#>   SVM-Type:  C-classification
#>   SVM-Kernel: radial
#>      cost:   1
#>
#> Number of Support Vectors: 2318
```

10.10 Evaluar la SVM radial

```
prediccion_svm_radial <- predict(
  modelo_svm_atus_radial,
  newdata = prueba_svm
)

matriz_svm_radial <- table(
  Real = prueba_svm$accidente_con_victimas,
  Predicho = prediccion_svm_radial
)

matriz_svm_radial
```

```
#>
#>          Predicho
#> Real          Solo daños Con víctimas
#> Solo daños      1180      296
#> Con víctimas    93       231
```

```
metricas_svm_radial <- metricas_clasificacion(
  real = prueba_svm$accidente_con_victimas,
  predicho = prediccion_svm_radial
)

metricas_svm_radial
```

```
#>   exactitud sensibilidad especificidad precision      f1
#> 1 0.7838889    0.712963    0.799458 0.4383302 0.5428907
```

10.11 Comparar los dos modelos

```
comparacion_svm <- bind_rows(
  transform(metricas_svm_lineal, modelo = "SVM lineal"),
  transform(metricas_svm_radial, modelo = "SVM radial")
) |>
  select(modelo, everything())

comparacion_svm
```

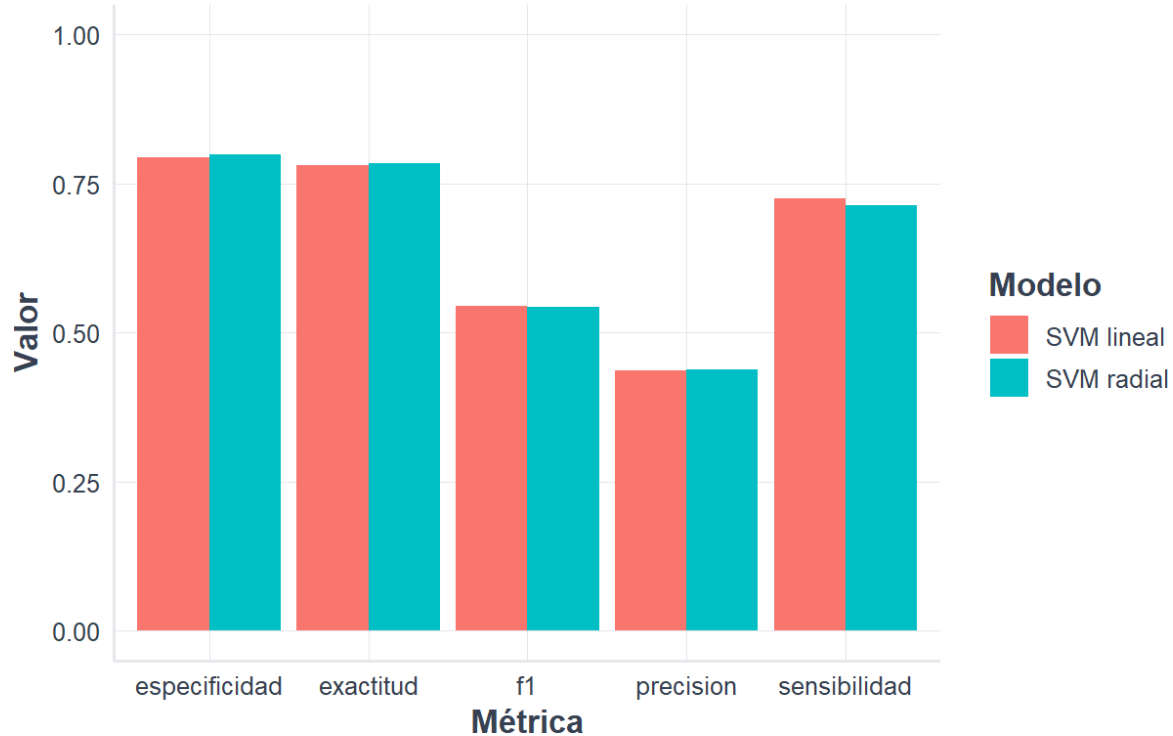
```
#>      modelo exactitud sensibilidad especificidad precision      f1
#> 1 SVM lineal 0.7816667    0.7253086    0.7940379 0.4359926 0.5446118
#> 2 SVM radial 0.7838889    0.7129630    0.7994580 0.4383302 0.5428907
```

```
comparacion_larga <- comparacion_svm |>
  tidyr::pivot_longer(
    cols = -modelo,
    names_to = "metrica",
    values_to = "valor"
  )

ggplot(
  comparacion_larga,
  aes(x = metrica, y = valor, fill = modelo)
) +
  geom_col(position = "dodge") +
  coord_cartesian(ylim = c(0, 1)) +
  labs(
    title = "Comparación de modelos SVM",
    subtitle = "Resultados sobre el conjunto de prueba",
    x = "Métrica",
    y = "Valor",
    fill = "Modelo"
  ) +
  tema_libro()
```

Comparación de modelos SVM

Resultados sobre el conjunto de prueba



10.12 Selección de hiperparámetros con validación cruzada

La función `tune.svm()` permite probar diferentes combinaciones de hiperparámetros. Para mantener un tiempo razonable se usa una cuadrícula pequeña.

```
set.seed(123)

muestra_ajuste <- entrenamiento_svm |>
  sample_n(min(3000, nrow(entrenamiento_svm)))

ajuste_svm <- tune.svm(
  accidente_con_victimas ~
    MES + ID_HORA + DIASEMANA + TIPACCID + CAUSAACCI,
  data = muestra_ajuste,
  kernel = "radial",
  cost = c(0.5, 1, 2),
  gamma = c(0.01, 0.03, 0.05),
  tunecontrol = tune.control(cross = 5),
  scale = TRUE
)

ajuste_svm$best.parameters
```

```
#> gamma cost
#> 2 0.03 0.5
```

```
mejor_modelo_svm <- ajuste_svm$best.model

prediccion_mejor_svm <- predict(
  mejor_modelo_svm,
  newdata = prueba_svm
)

metricas_mejor_svm <- metricas_clasificacion(
  real = prueba_svm$accidente_con_victimas,
  predicho = prediccion_mejor_svm
)

metricas_mejor_svm
```

```
#> exactitud sensibilidad especificidad precision      f1
#> 1 0.8533333      0.1851852              1      1 0.3125
```

Validación sin fuga de información

Los hiperparámetros deben seleccionarse usando exclusivamente los datos de entrenamiento. El conjunto de prueba se reserva para la evaluación final.

10.13 Ventajas y limitaciones

10.13.1 Ventajas

- puede construir fronteras lineales y no lineales;
- funciona bien en espacios con muchas variables;
- el margen máximo favorece la generalización;
- utiliza principalmente los vectores de soporte para definir la frontera;
- permite ponderar clases desbalanceadas.

10.13.2 Limitaciones

- puede ser lenta con conjuntos de datos muy grandes;
- requiere elegir kernel e hiperparámetros;
- sus resultados son menos interpretables que los de una regresión logística o un árbol pequeño;
- la estandarización de variables numéricas es importante;
- una búsqueda extensa de hiperparámetros puede consumir muchos recursos.

10.14 Recomendaciones prácticas

1. Comience con una SVM lineal como referencia.
2. Estandarice las variables numéricas.
3. Use validación cruzada para seleccionar `cost` y `gamma`.

4. Revise sensibilidad, especificidad, precisión y F1, no solo exactitud.
5. Utilice pesos de clase cuando exista desbalance.
6. Trabaje inicialmente con una muestra si el conjunto es muy grande.
7. Evalúe el modelo final una sola vez sobre datos de prueba no utilizados durante el ajuste.

10.15 Actividad guiada

Modifique el código para comparar:

- `cost = 0.1, 1` y `10` en la SVM lineal;
- `gamma = 0.01, 0.05` y `0.10` en la SVM radial;
- modelos con y sin `class.weights`;
- muestras de 3 000 y 6 000 accidentes.

Registre para cada modelo:

- número de vectores de soporte;
- tiempo de entrenamiento;
- exactitud;
- sensibilidad;
- especificidad;
- precisión;
- valor F1.

10.16 Ejercicios

1. Explique con sus propias palabras qué es un vector de soporte.
2. ¿Qué diferencia existe entre una frontera lineal y una frontera construida con kernel radial?
3. ¿Qué ocurre generalmente cuando `cost` es demasiado grande?
4. ¿Por qué es importante estandarizar variables numéricas?
5. Entrene una SVM lineal sin pesos de clase y compare su sensibilidad con el modelo ponderado.
6. Amplíe la cuadrícula de validación cruzada y explique si el mejor modelo cambia.
7. Compare la mejor SVM con la regresión logística y Random Forest estudiados anteriormente.
8. Analice si una mejora pequeña en exactitud justifica una pérdida importante de interpretabilidad.

10.17 Conclusiones

Las máquinas de vectores de soporte buscan una frontera con margen amplio y se apoyan en un subconjunto de observaciones denominado vectores de soporte. Mediante kernels pueden representar relaciones no lineales y producir modelos predictivos potentes.

Sin embargo, una SVM requiere seleccionar cuidadosamente sus hiperparámetros y evaluar su desempeño con datos no utilizados durante el entrenamiento. En problemas con clases desbalanceadas, la ponderación de clases y el análisis conjunto de sensibilidad, especificidad, precisión y F1 son fundamentales.

Fuente de los datos de aplicación: elaboración propia con datos del INEGI, Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024.

Referencias

Instituto Nacional de Estadística y Geografía. 2025. «Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS), 2024». Instituto Nacional de Estadística y Geografía. <https://www.inegi.org.mx/programas/accidentes/>.

Instituto Nacional de Estadística y Geografía. 2026. «Términos de libre uso de la información del INEGI». <https://www.inegi.org.mx/inegi/terminos.html>.

Reconocimiento de la fuente de datos

Los datos de accidentes empleados en este libro proceden de la **Estadística de Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas (ATUS)** del Instituto Nacional de Estadística y Geografía (Instituto Nacional de Estadística y Geografía 2025).

Los datos originales pertenecen al INEGI. La limpieza, transformación, selección de variables, gráficas, modelos de aprendizaje automático e interpretaciones son elaboración del autor y no constituyen resultados oficiales ni implican el aval del Instituto.